



ALMA MATER STUDIORUM
UNIVERSITÀ DI BOLOGNA

Introduction to Transformers

Prof. Luca Benini

Outline

- AI Models/Architectures for Natural Language Processing
- The limitations of RNNs and LSTMs
- The Transformers as a solution
- Encoder
 - Input Embedding and Positional Encoding
 - Self-Attention and Multi-Head Attention
- Decoder
 - Output Embedding
 - Masked Multi-Head Attention
 - Multi-Head Attention with Encoder's K,V values
 - Linear layers and softmax

Training

- Inference
- Applications
- Efficient Transformers Architectures

Natural Language Processing

Google Translate – Machine Translation

ENGLISH - DETECTED

ENGLISHSPANISHFRENCH

×

Have you found it difficult to deploy neural networks on mobile devices and IoT devices? Have you ever found it too slow to train neural networks? This course is a deep dive into efficient machine learning techniques that enable powerful deep learning applications on resource-constrained devices. Topics cover efficient inference techniques, including model compression, pruning, quantization, neural architecture search, distillation; and efficient training techniques, including gradient compression and on-device transfer learning; followed by application-specific model optimization techniques for videos, point cloud and NLP; and efficient quantum machine learning. Students will get hands-on experience implementing deep learning applications on microcontrollers, mobile phones and quantum machines with an open-ended design project related to mobile AI.



862 / 5,000

↔

CHINESE (SIMPLIFIED)

ENGLISHSPANISH

☆

您是否发现很难在移动设备和物联网设备上部署神经网络？你有没有发现训练神经网络太慢了？本课程深入探讨有效的机器学习技术，这些技术可在资源受限的设备上实现强大的深度学习应用。主题涵盖高效推理技术，包括模型压缩、剪枝、量化、神经架构搜索、蒸馏；和高效的训练技术，包括梯度压缩和设备迁移学习；其次是针对视频、点云和 NLP 的特定应用模型优化技术；和高效的量子机器学习。学生将通过与移动 AI 相关的开放式设计项目获得在微控制器、手机和量子机器上实施深度学习应用程序的实践经验。

Nín shìfǒu fāxiàn hěn nán zài yídòng shèbèi hé wù liánwǎng shèbèi shàng bùshǔ shénjīng wǎngluò? Nǐ yǒu méiyǒu fāxiàn xùnlìan shénjīng wǎngluò tài mànle? Běn kèchéng shēnrù tàntǎo yǒuxiào de jīqì xuéxí jìshù, zhèxiē jìshù kě zài zīyuán shòu xiàn de

[Show more](#)



Natural Language Processing

Chat Bot – Question Answering

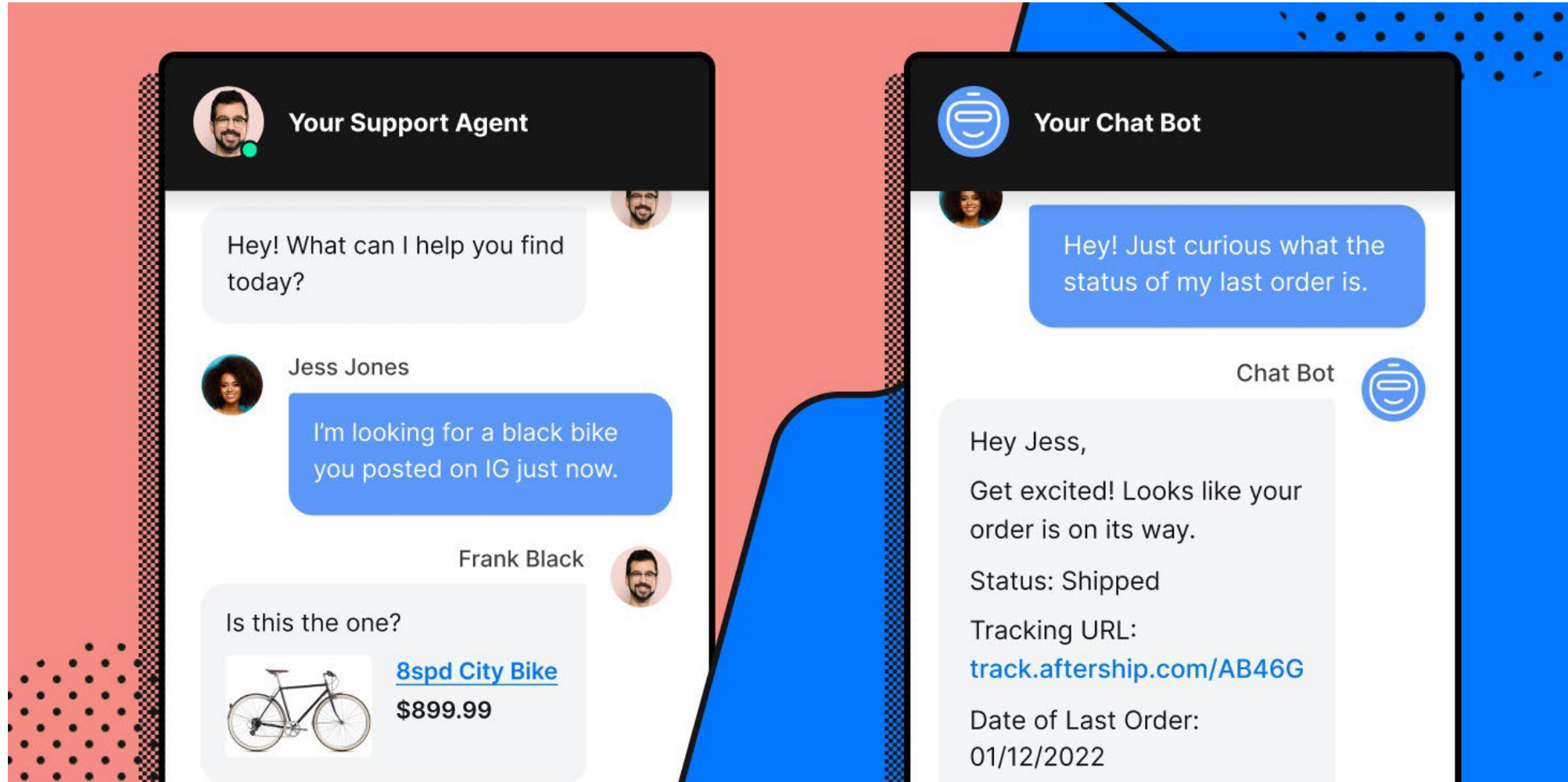
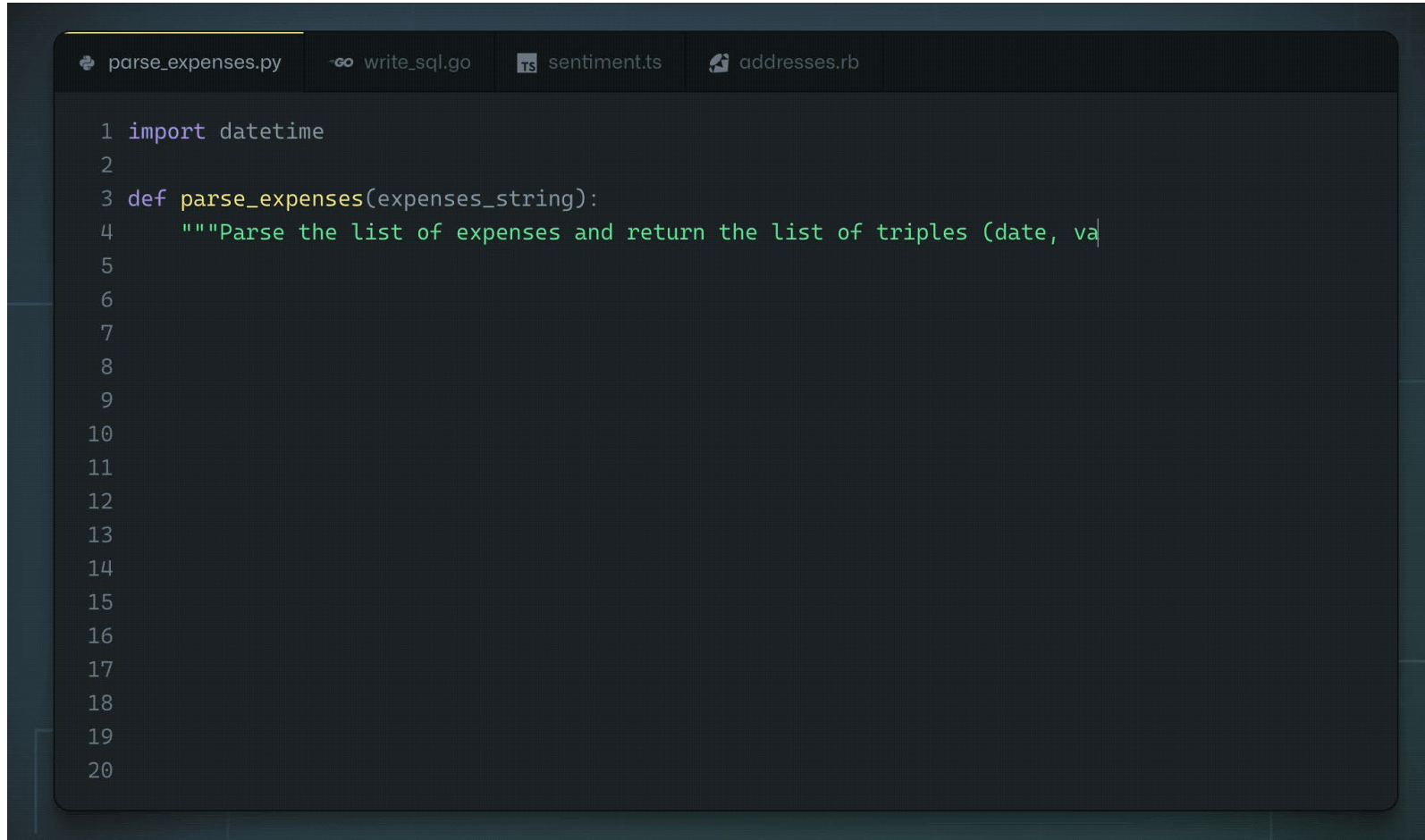


Image credit: <https://www.gorgias.com/blog/chatbot-vs-live-chat>

Natural Language Processing

GitHub Copilot – Language Modeling



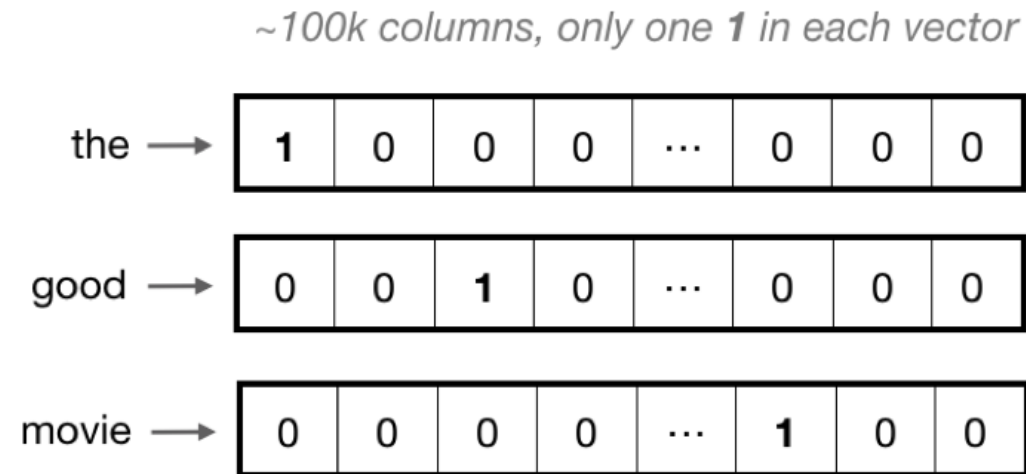
The image shows a screenshot of a code editor with a dark theme. At the top, there are four tabs: 'parse_expenses.py' (active), 'write_sql.go', 'sentiment.ts', and 'addresses.rb'. The main editor area displays a Python function named 'parse_expenses' that takes 'expenses_string' as an argument. The function's docstring indicates it should parse a list of expenses and return a list of triples (date, value, category). The code is as follows:

```
1 import datetime
2
3 def parse_expenses(expenses_string):
4     """Parse the list of expenses and return the list of triples (date, va
5
6
7
8
9
10
11
12
13
14
15
16
17
18
19
20
```

How to Represent Words for NLP

One-Hot Encoding

- **Key idea:** Representing each word as a vector that has as many values in it as there are words in the vocabulary.
 - Each column in a vector represents one possible word in a vocabulary



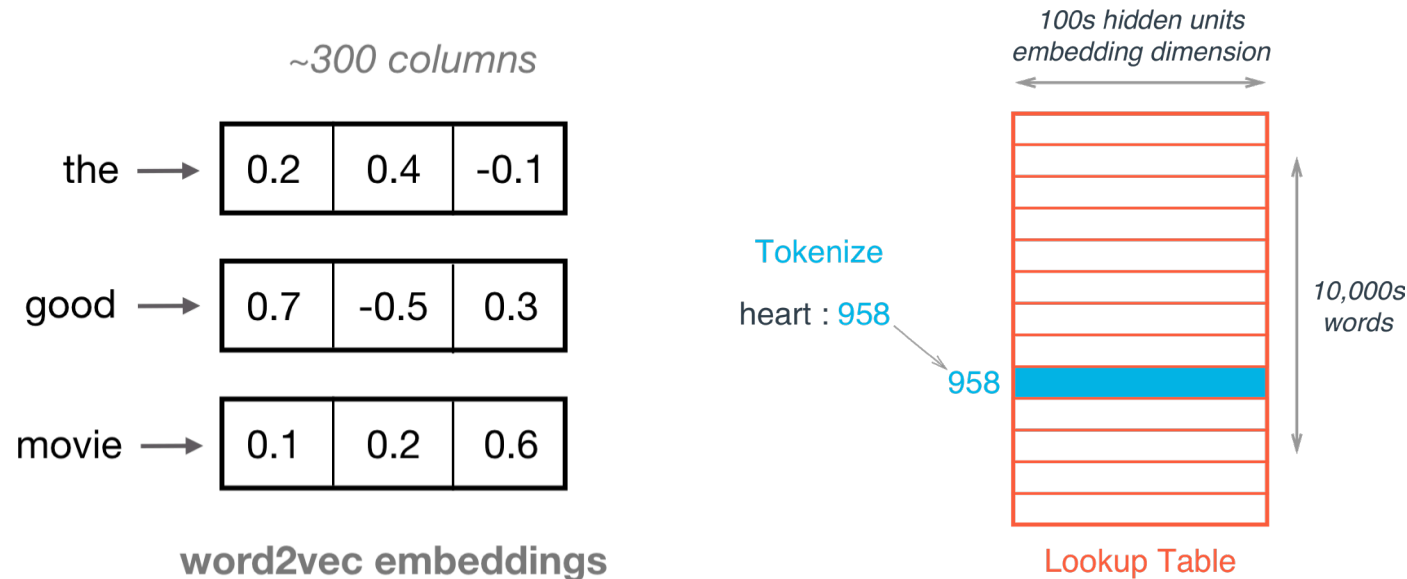
- Very sparse and inefficient representation → For large vocabularies, these vectors can get very long, and they contain all 0's except for one value → Manageable only for small vocabulary
- It does not capture semantic relationships between words → NOT USED

https://cezannec.github.io/CNN_Text_Classification/

How to Represent Words for NLP

Word embeddings

- **Key idea:** Representing each word as a vector, but **denser than one hot encoding!**

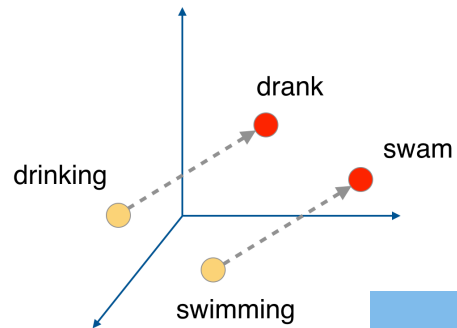


- How do you assign vectors to words (tokens)? **word2vec**
- **Key idea 2 – train word2vec by assigning similar vectors to words that are often in the same context**

How to Represent Words for NLP → word2vec

Word embeddings with word2vec

- Use cosine similarity as similarity metric



$$\text{cosine similarity} = S_C(A, B) := \cos(\theta) = \frac{\mathbf{A} \cdot \mathbf{B}}{\|\mathbf{A}\| \|\mathbf{B}\|} = \frac{\sum_{i=1}^n A_i B_i}{\sqrt{\sum_{i=1}^n A_i^2} \cdot \sqrt{\sum_{i=1}^n B_i^2}}$$

$$u^T v = u \cdot v = \sum_{i=1}^n u_i v_i$$

dotp again if A,B have norm 1

- Assign vectors using gradient to minimize a loss function on a (training) text corpus:
- T = text length, m = window size, $P(w_{t+j}|w_t)$ = the probability to get context word given target word.

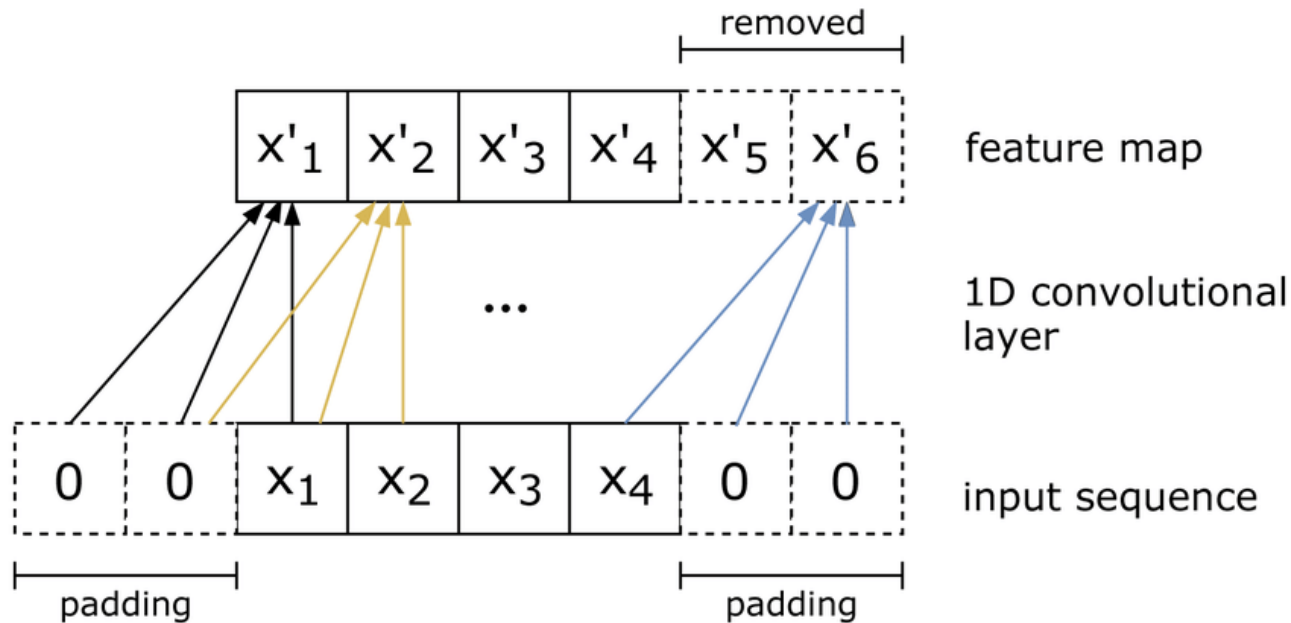
$$J = - \sum_{t=1}^T \sum_{-m \leq j \leq m} \log(P(w_{t+j}|w_t))$$

$$p(\text{context word}/\text{target word}) = \frac{\exp(u_{\text{target}}^T v_{\text{context}})}{\sum_{w=1}^{\text{Words}} (\exp(u_{\text{target}}^T v_w))}$$

P will be “large” if words that are often found at distance $2m$ or less have “similar vectors”
(larger dotp than words that are at larger distances in the corpus)

Pre-Transformers Era for NLP

Convolutional Neural Networks (CNNs) – 1d convolutions



this →	0.2	0.4	-0.3
movie →	0.1	0.2	0.6
has →	-0.1	0.4	-0.1
amazing →	0.7	-0.5	0.4
diverse →	0.1	-0.2	0.1
characters →	0.6	-0.3	0.8

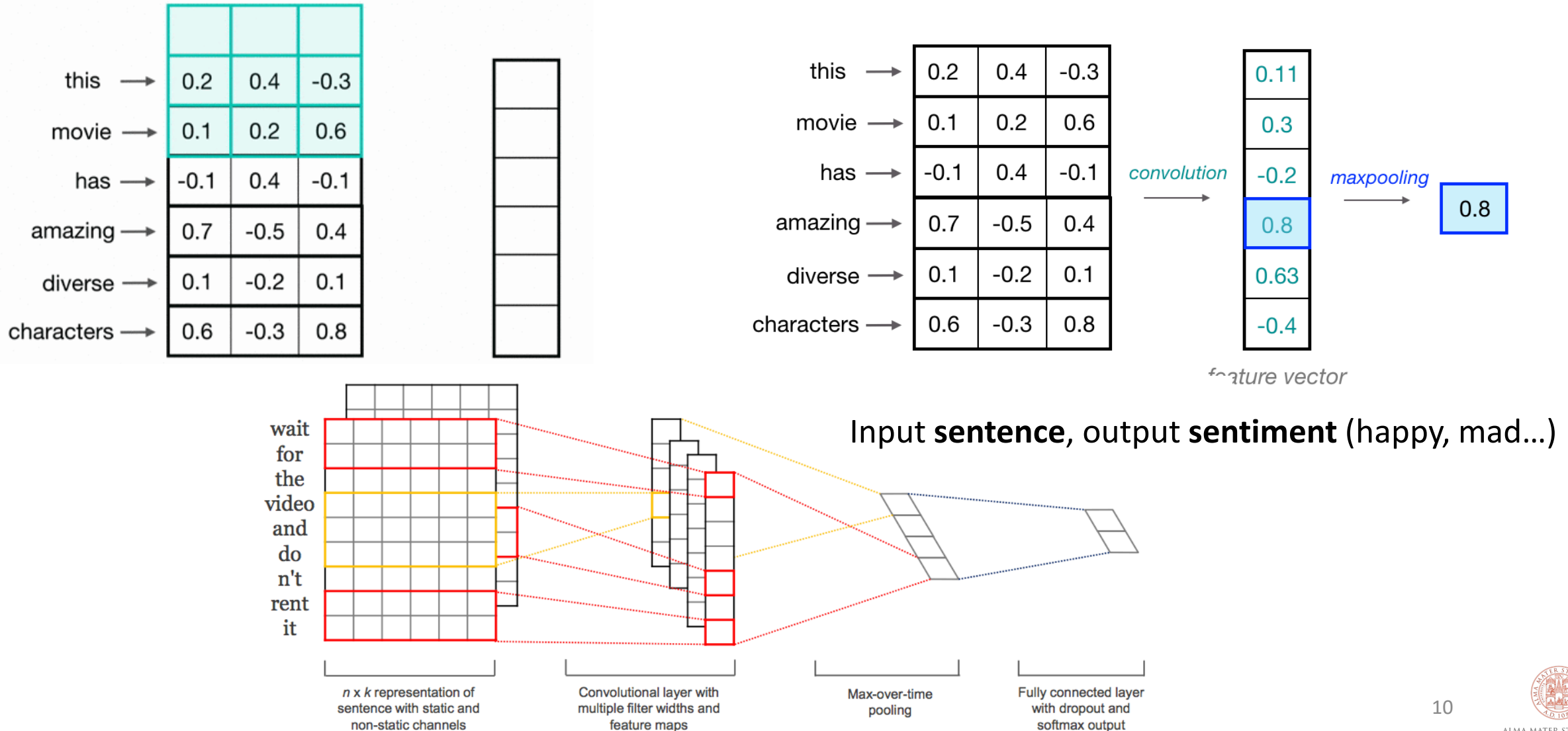
the →	0.2	0.4	-0.1
good →	0.7	-0.5	0.3
movie →	0.1	0.2	0.6

0.5	0.4	0.7
0.2	-0.1	0.3

convolutional kernel

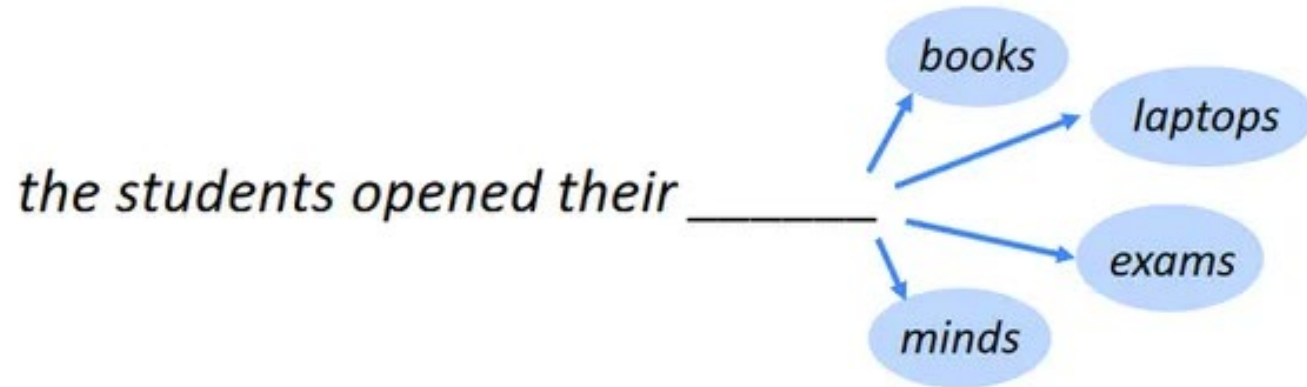
Pre-Transformers Era for NLP → CNNs for text classification

Convolutional Neural Networks (CNNs) – 1d convolutions



Pre-Transformers Era for NLP: enter next word prediction

Given a text context, predict the next (most likely) word



Very “fundamental tasks:

1. Can be used autoregressively → feed prediction to the input to predict “many words”, i.e. sentences
2. Can be used for many end to end tasks: e.g. sentiment analysis

Analyze the sentiment of the following sentence: “I absolutely loved the new pizza place in town!”



“positive”



Can you analyze the sentiment of the following sentence: "I absolutely loved the new pizza place in town!"

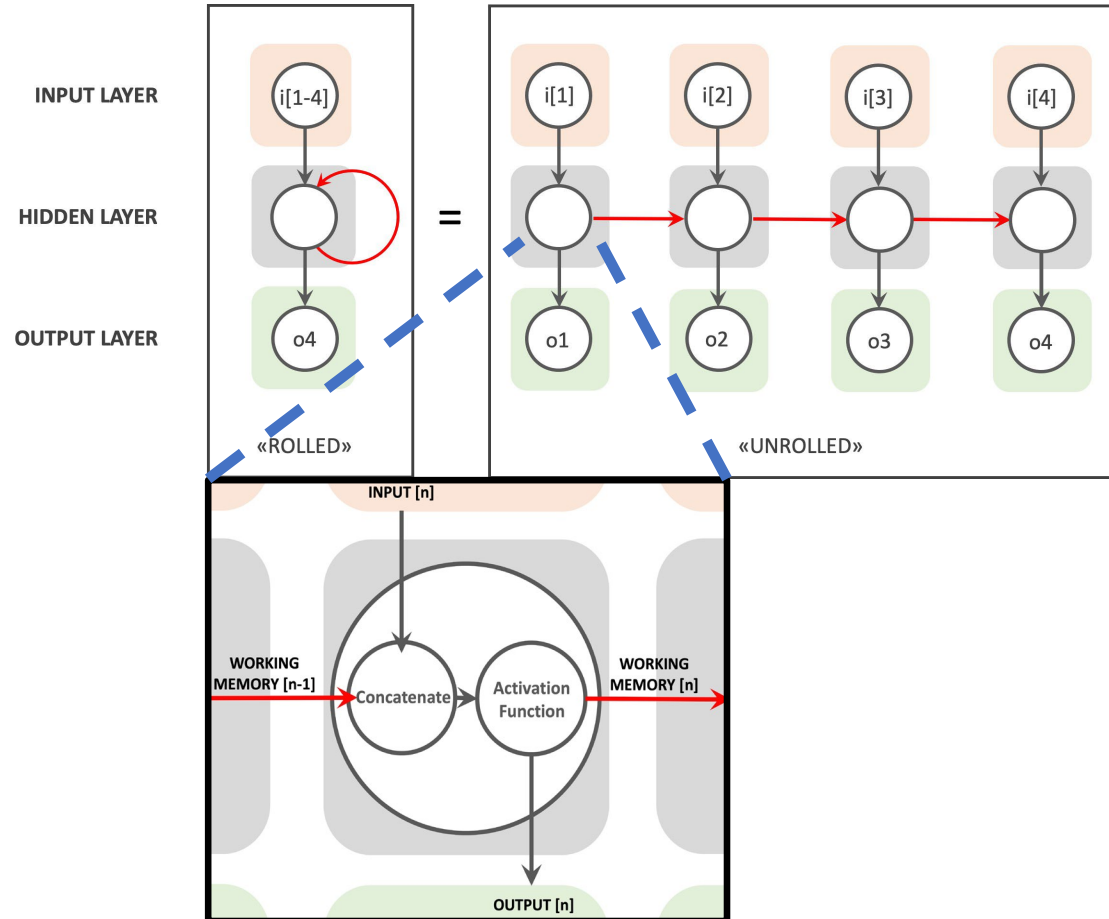


The sentence "I absolutely loved the new pizza place in town!" expresses a positive sentiment. The use of the word "loved" and the exclamation mark indicates a strong positive emotion towards the new pizza place.

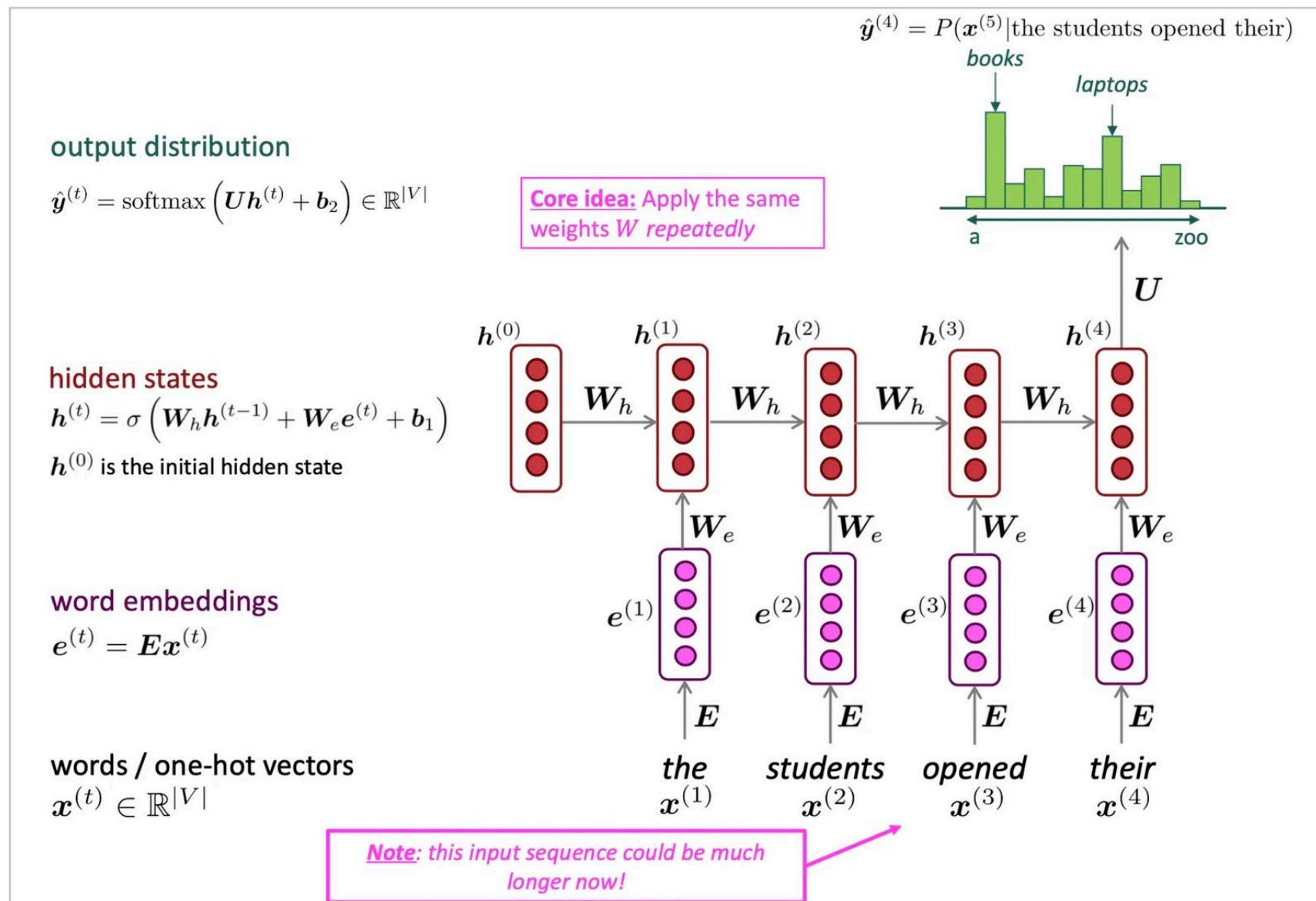
Pre-Transformers Era for NLP → stateful networks

Recurrent Neural Networks (RNNs)

- Specifically designed to process sequential data (time series, NLP)

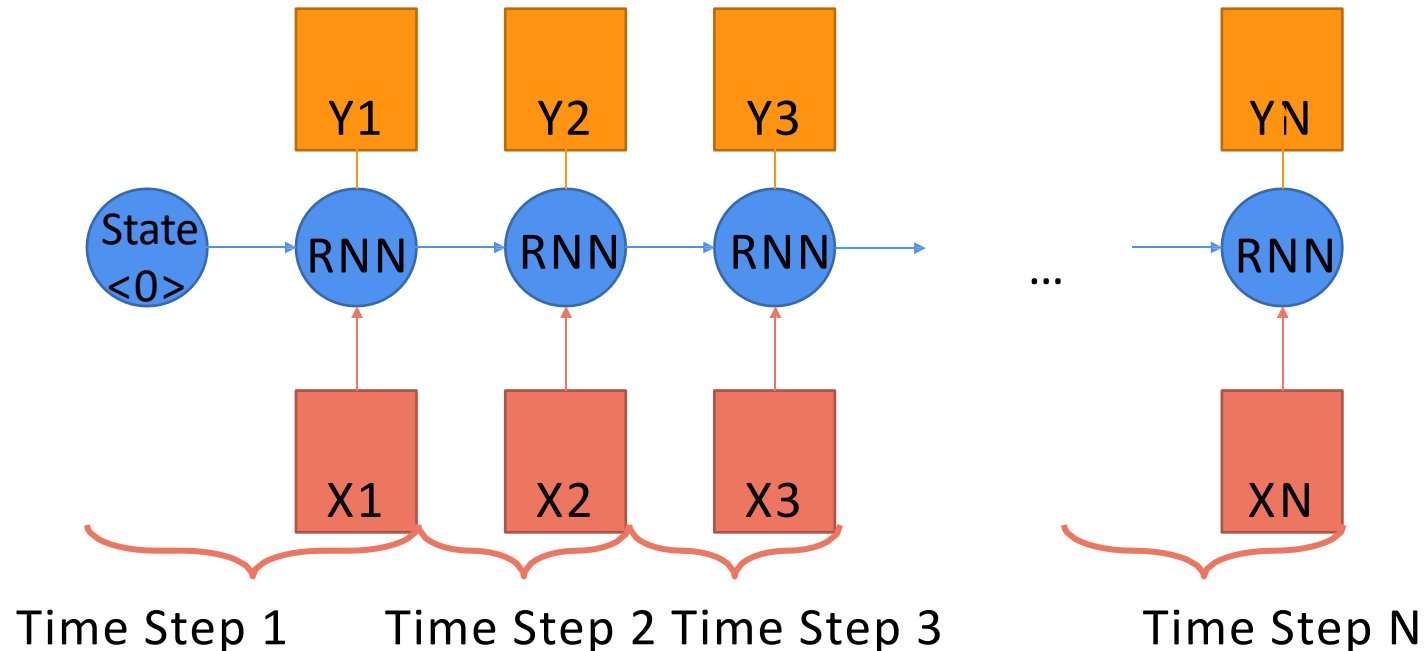


Recurrent Neural Networks (RNNs)



Pre-Transformers Era for NLP

Issues and Limitations of Recurrent Neural Networks (RNNs)



- Slow computation for long sequences → limited scalability
- There is strict dependency across tokens → limited scalability
- the "working memory" struggles to retain long-term dependencies → Difficulty in accessing information from long time ago
- Vanishing Gradient problem...
- More advanced RNN → LSTM etc. or...

Transformers! Two key components

- **Attention mechanism (not really new)**
 - This is the core of transformers
- **Word embeddings + positional encoding (not really new)**
 - This replaces the one-hot encoding of RNNs for a more dense and meaningful representation of words

But the combination has been transformative!

How to Capture Useful Relations Among Tokens?

Pay attention to some specific tokens (words)!

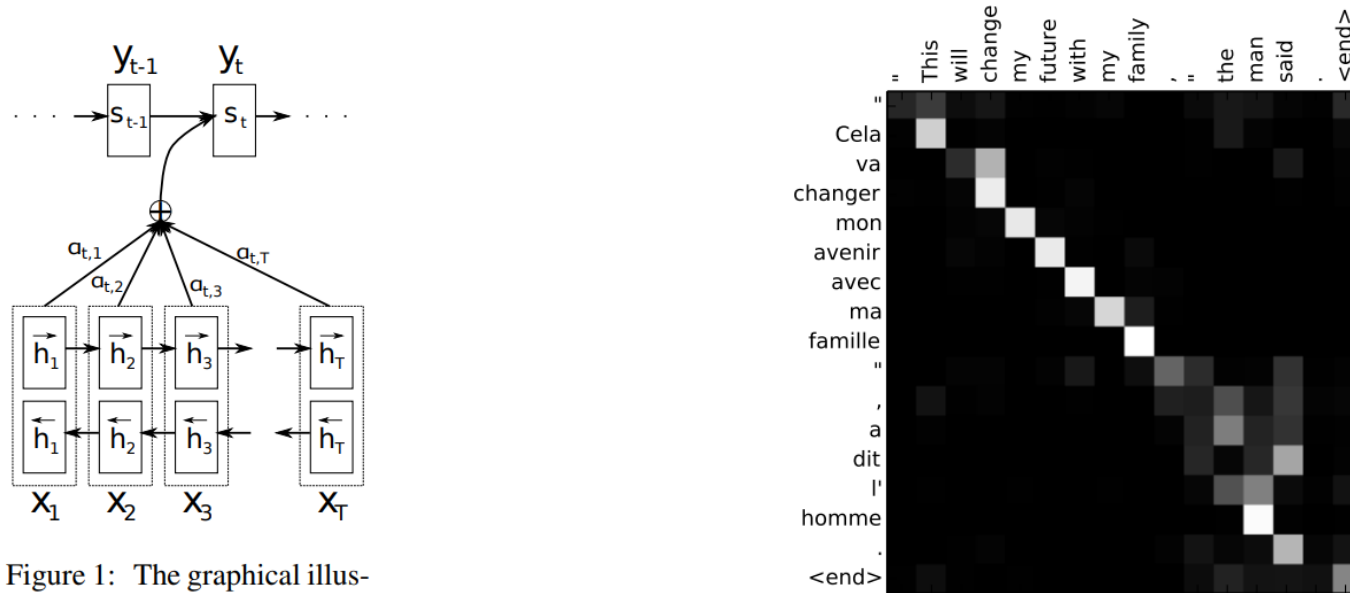
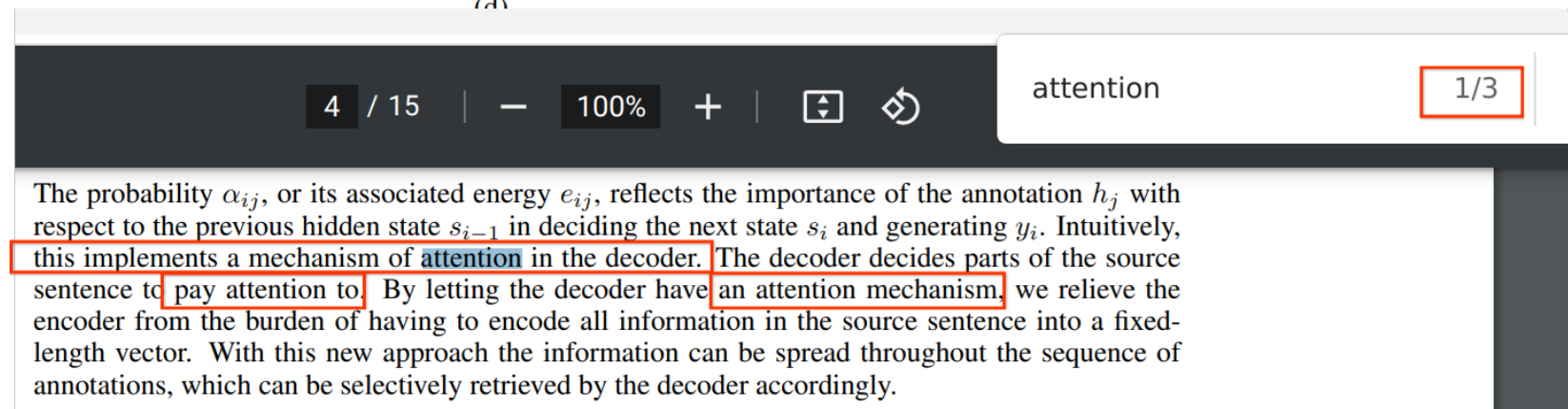


Figure 1: The graphical illustration of the proposed model trying to generate the t -th target word y_t given a source sentence $(x_1, x_2, \dots, x_T)$.

2014, Dzmitry Bahdanau,
KyungHyun Cho

The "Attention" allows to focus on different parts of a sentence simultaneously rather than sequentially (like in RNNs) → capturing dependencies and relations regardless of the distance → CONTEX-SENSITIVE



Attention is All You Need

Attention is a function similar to a "soft" **kv** dictionary lookup:

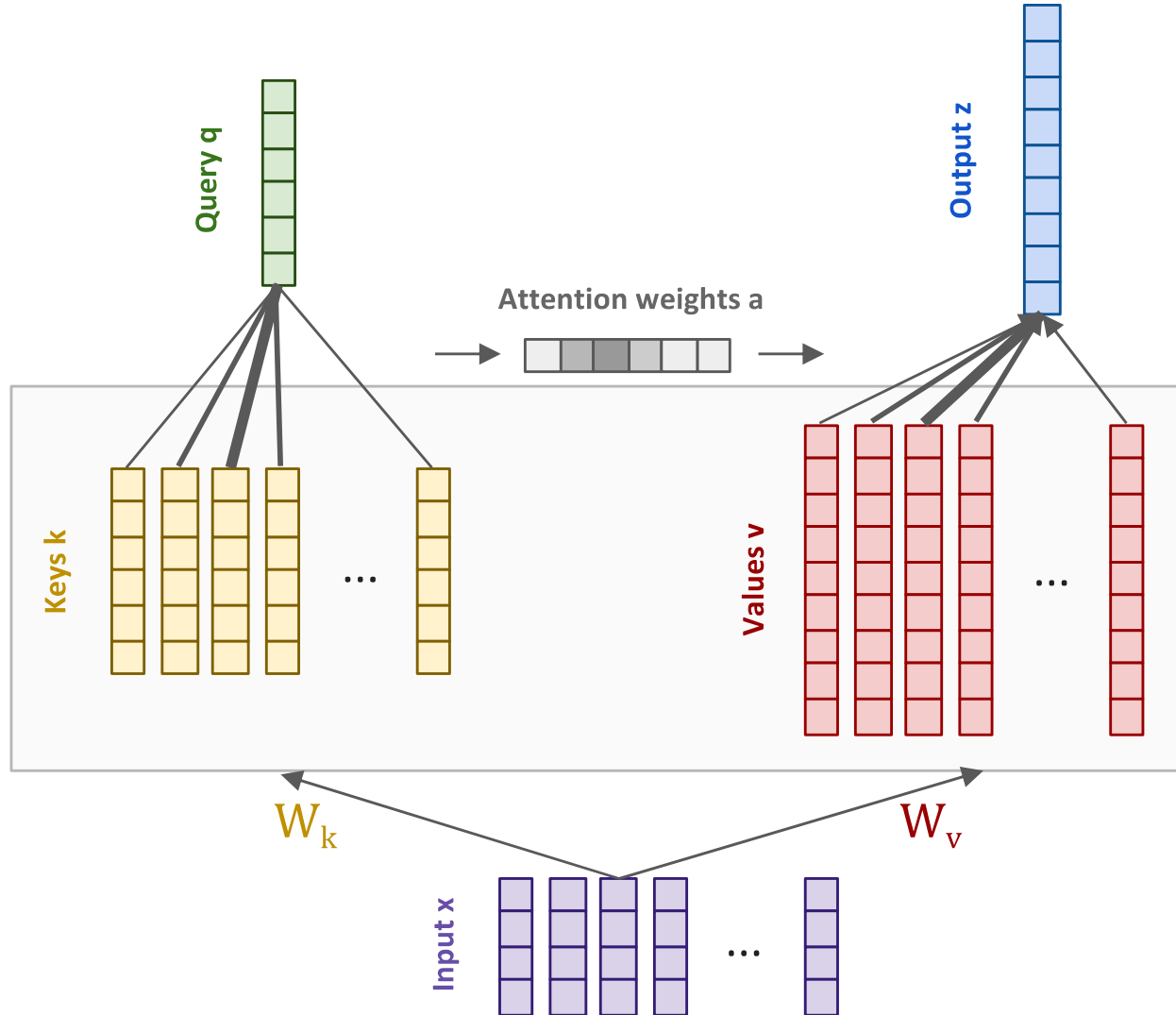
- "key" and "values" are reminiscent of a dictionary in SW, with keys mapping to values
- Look-up table
- `Table = {"key0": "value0", "key1": "Value1", ...}`
- Query
- `Table["key0"] → "value0"`
- In "attention" instead of a single entry we return a weighted combination of entries.
- Attention weights $a_{1:N}$ are query-key similarities:
 - $\hat{a}_i = \langle \mathbf{q}, \mathbf{k}_i \rangle$
 - Normalized via softmax: $a_i = e^{\hat{a}_i} / \sum_j e^{\hat{a}_j}$
- **Output** $\mathbf{z}$ is attention-weighted average of **values** $\mathbf{v}_{1:N}$:
 - $\mathbf{z} = \sum_i \hat{a}_i \mathbf{v}_i = \hat{\mathbf{a}} \cdot \mathbf{v}$

2017, Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N. Gomez, Łukasz Kaiser, Illia Polosukhin

Attention is All You Need

Attention is a function similar to a "soft" **kv** dictionary lookup:

- Usually, **k** and **v** are derived from the same input **x**:
 - $k = W_k \cdot x$
 - $v = W_v \cdot x$
- The query **q** can come from a separate input **y**:
 - $q = W_q \cdot y$
- Or from the same input **x**! Then we call it "self attention":
 - $q = W_q \cdot x$

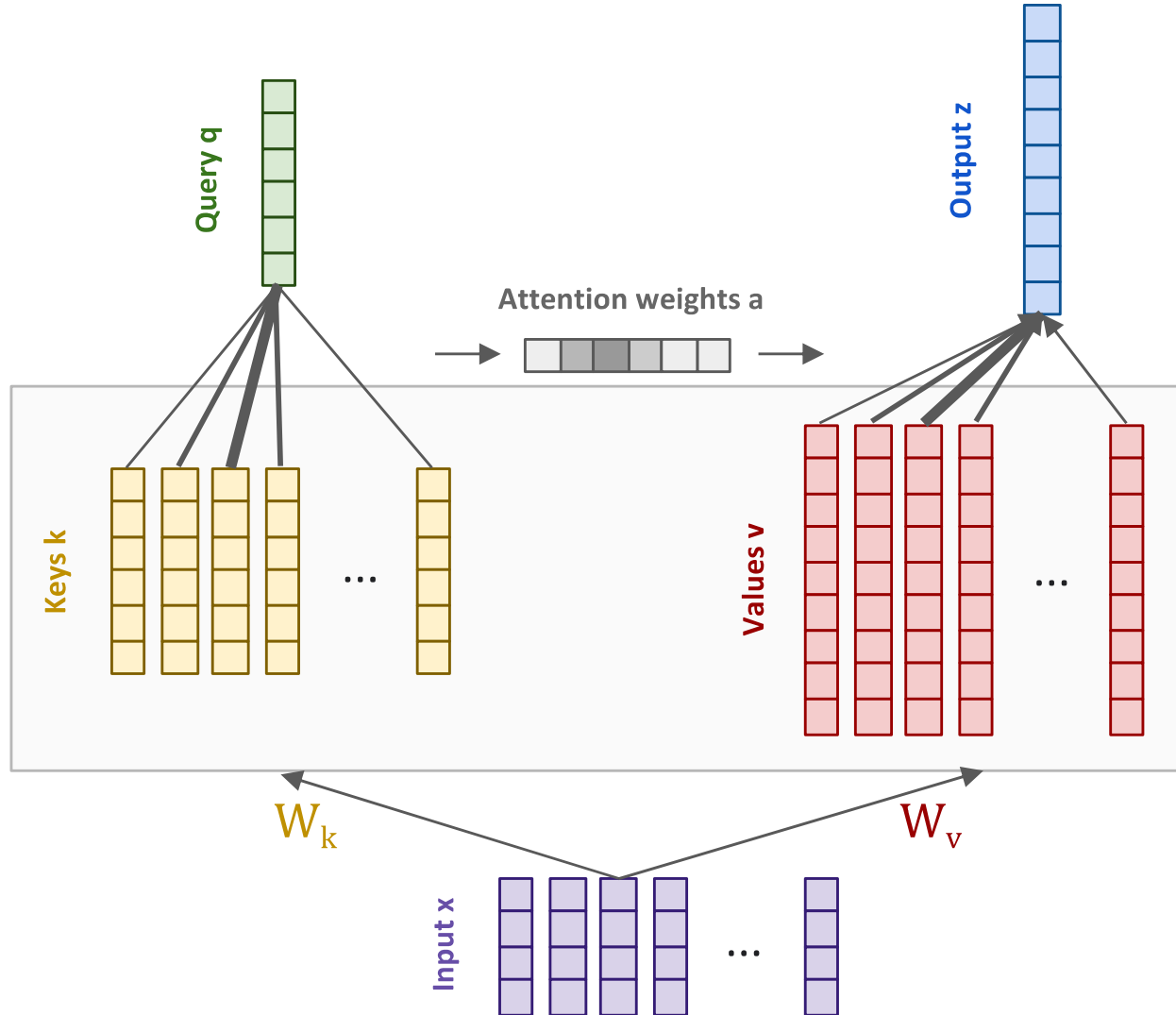


2017, Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N. Gomez, Łukasz Kaiser, Illia Polosukhin

Attention is All You Need

Attention is a function similar to a "soft" **kv** dictionary lookup:

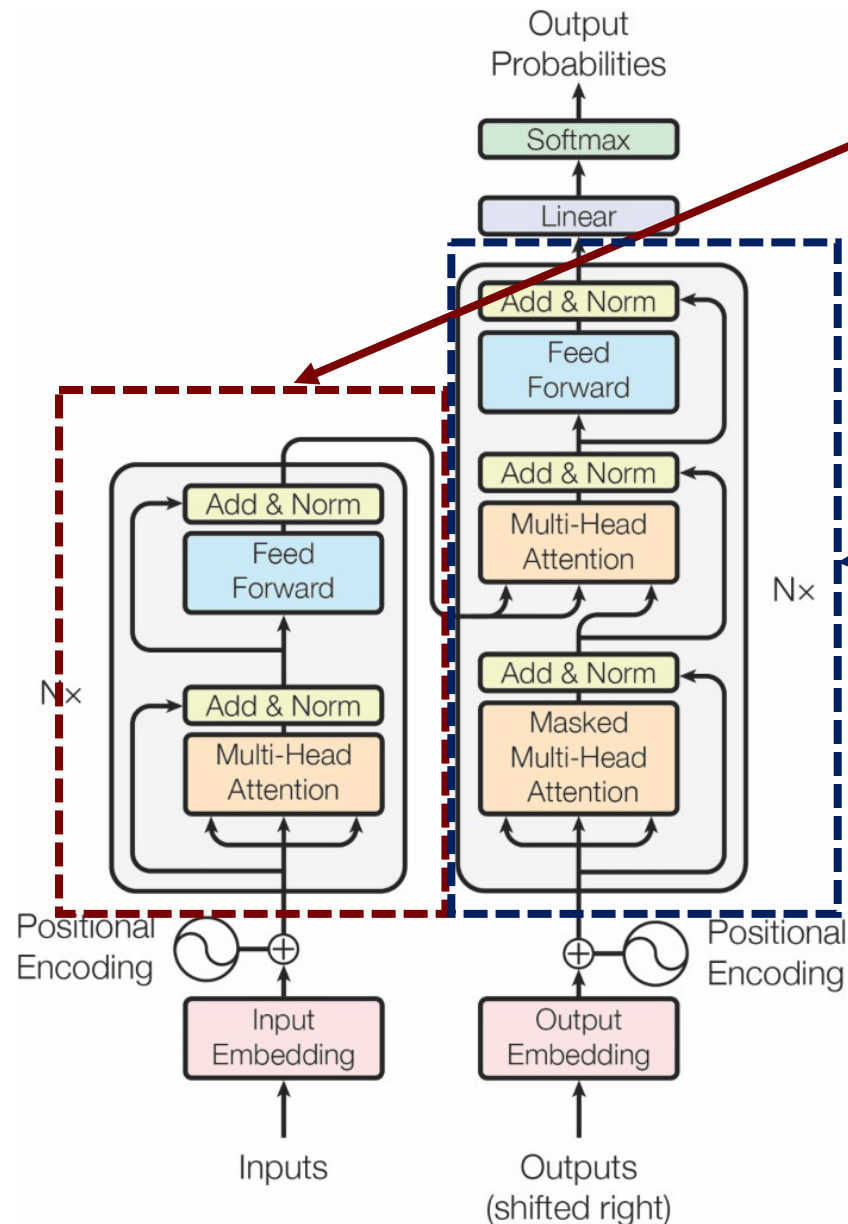
- Usually, **k** and **v** are derived from the same input **x**:
 - $k = W_k \cdot x$
 - $v = W_v \cdot x$
- The query **q** can come from a separate input **y**:
 - $q = W_q \cdot y$
- Or from the same input **x**! Then we call it "self attention":
 - $q = W_q \cdot x$



2017, Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N. Gomez, Łukasz Kaiser, Illia Polosukhin

The Architecture of Transformer

The Transformer



The Transformer is composed of:

- N Encoder Blocks, each of which has three sub-layers:
 - **Multi-head self-attention** mechanism
 - Position-wise **fully connected feed-forward** network
 - **Layer normalization** and **residual connection** layers
- N Decoder blocks, each of which has four sub-layers:
 - **Masked Multi-head self-attention** mechanism
 - **Multi-head attention** over the output of the encoder stack
 - Position-wise **fully connected feed-forward** network
 - **Layer normalization** and **residual connection** layers
- Both Encoders and Decoders are preceded by **embeddings** and **positional encoding** mechanisms
- The decoder generates the output sequence of symbols one element at time in an auto-regressive manner

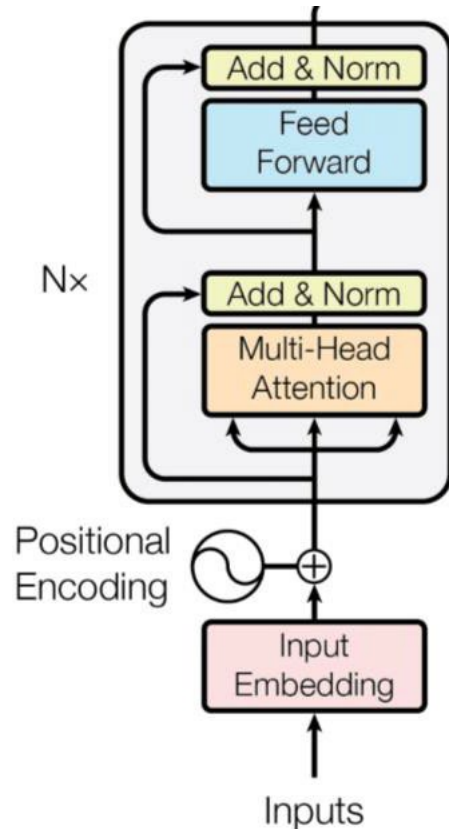
Transformer's Main Parameters

- **Seq** = sequence length → Number of tokens in the input sequence, they are processed in parallel in a Transformer Architecture
- d_{model} = size of the embedding vector → size of the embedding vector for each token in the sequence → It represents the dimensionality of the feature space for each token [This parameter depends on the specific transformer model we use]
- **h** = number of heads → number of attention heads in the multi-head attention mechanism.
- $d_k - d_v = d_{\text{model}} / h$ → dimensionality of key and value vectors for each head in the multi-head attention.

There are other parameters that we do not cover in this lecture..

The Transformers' Encoder

Encoding / Encoder

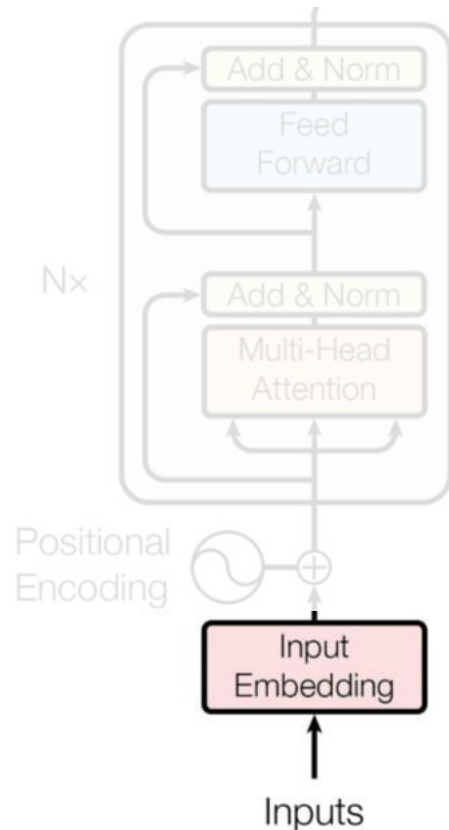


Since input and output shapes are identical, we can stack N such blocks.

Encoder output is a "heavily processed" (think: "high level, contextualized") version of the input tokens, i.e. a sequence.

This has nothing to do with the requested output yet (think: translation). That comes with the decoder.

The Input Embedding



an **input embedding** is like a translator that converts each word into a form that a transformer can work with to learn the relationships between words and understand the text.

1. Input tokenization: Input text is cut into pieces . Can be characters, words, or, more in general, “tokens”
2. Input IDs: Tokens are mapped to some numbers that represent their position in a “vocabulary space”. → Once the vocabulary is set, input IDs will not change
3. Embedding: We map the input IDs to a vector of d_{model} elements that represents the meaning of that token (word) → These are trainable parameters that the model will learn from the training dataset → They change and adapt to the “context”

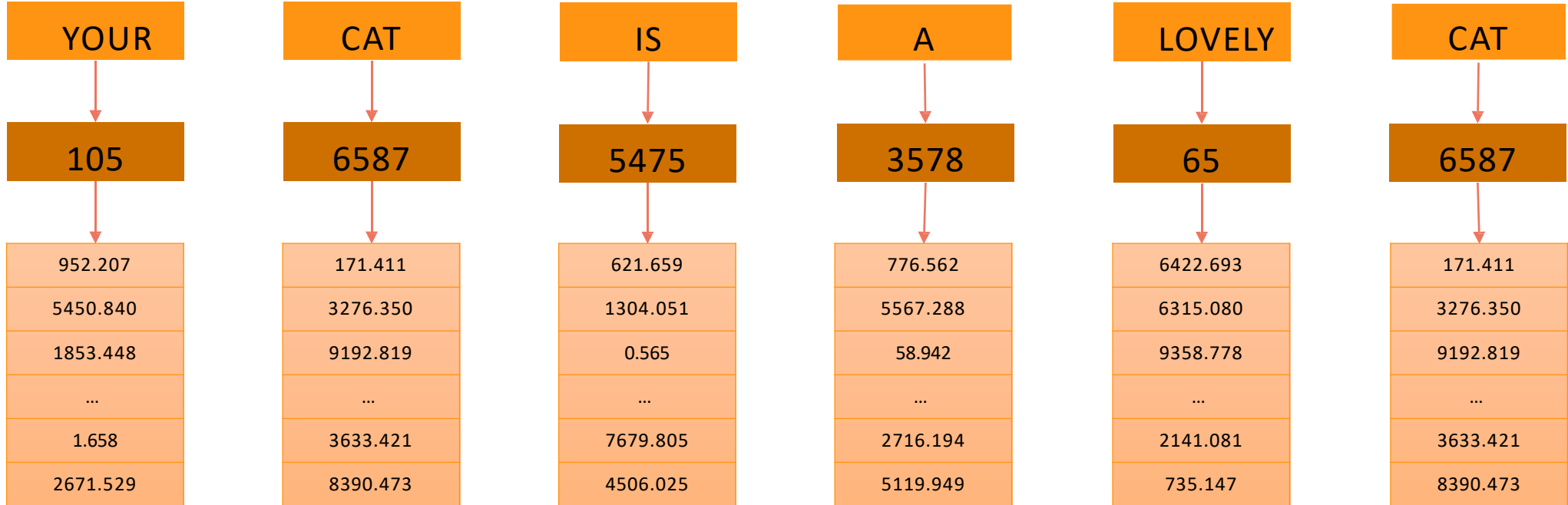
The Embedding is the INPUT to our Transformer model.

What is an Input Embedding?

Original
sentence
(tokens)

"<SOS> YOUR CAT IS A LOVELY CAT <EOS>"

Tokenization

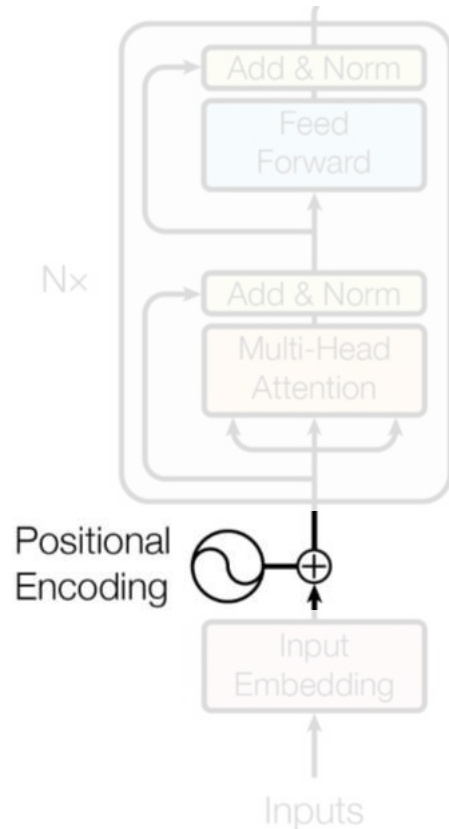


Input IDs
(position in the
vocabulary)

Embedding
(vector of size
 $d_{\text{model}} = 512$)

We define $d_{\text{model}} = 512$, which represents the size of the embedding vector of each word

The Positional Encoding



- We want each word to carry some information about its position in the sentence
 - For humans it's simple, we “visualize” the text and clearly recognize “token” positions
 - We want the model to treat tokens that appear close to each other as “CLOSE” and words that are distant as “distant”
 - We want the positional encoding to represent a pattern that can be learned by the model
- This is only computed once and re-used for every sentence both in training and inference
- Note: Needed in this model since we do not have anymore the sequential structure of RNNs

What is Positional Encoding?

Original sentence

YOUR

CAT

IS

A

LOVELY

CAT

Embedding
(vector of size 512)

952.207

5450.840

1853.448

...

1.658

2671.529

171.411

3276.350

9192.819

...

3633.421

8390.473

621.659

1304.051

0.565

...

7679.805

4506.025

776.562

5567.288

58.942

...

2716.194

5119.949

6422.693

6315.080

9358.778

...

2141.081

735.147

171.411

3276.350

9192.819

...

3633.421

8390.473

Position Embedding
(vector of size 512).
Only computed once and reused for every sentence during training and inference.

+

+

+

+

+

+

...

...

...

...

...

...

1664.068

8080.133

2620.399

...

9386.405

3120.159

...

...

...

...

...

...

...

...

...

...

...

...

...

...

...

...

...

...

1281.458

7902.890

912.970

3821.102

1659.217

7018.620

=

=

=

=

=

=

...

...

...

...

...

...

1835.479

11356.483

11813.218

...

13019.826

11510.632

...

...

...

...

...

...

...

...

...

...

...

...

...

...

...

...

...

...

1452.869

11179.24

10105.789

...

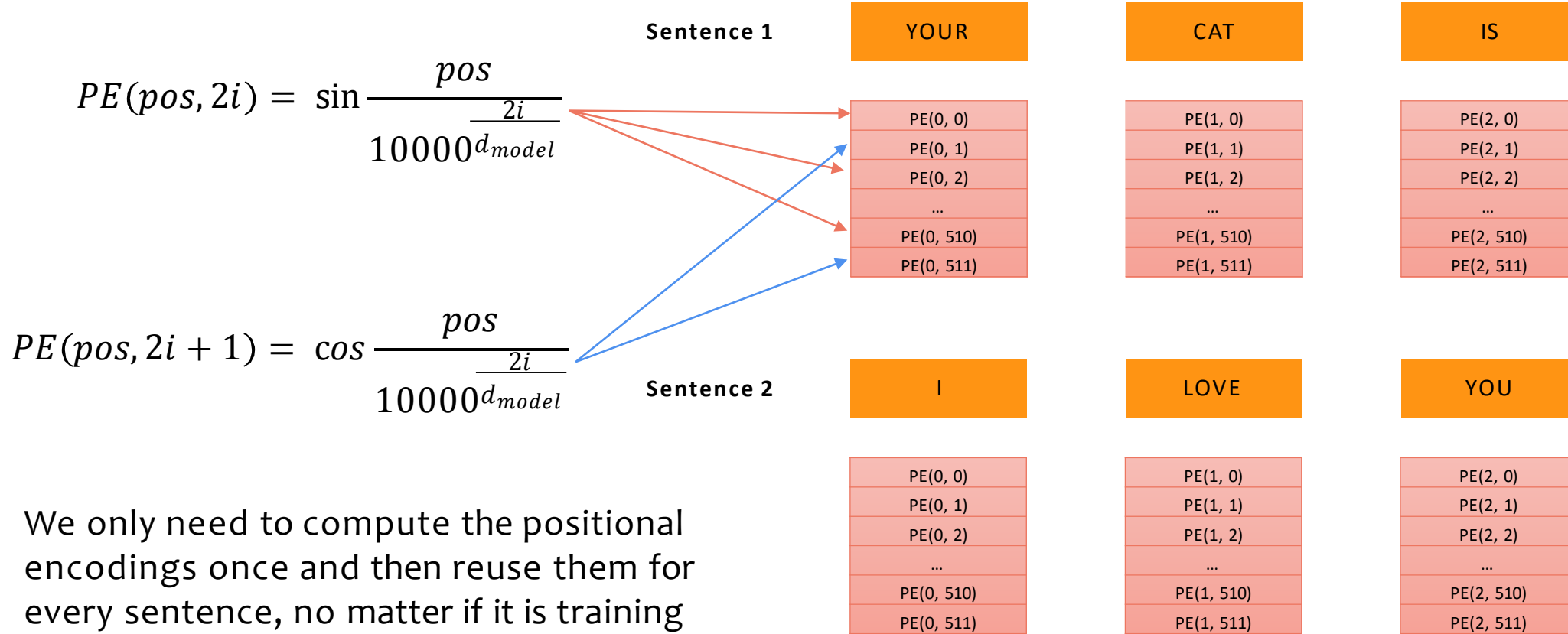
5292.638

15409.093

Encoder Input
(vector of size 512)

Credits: Transformer from scratch, Umar Jamil

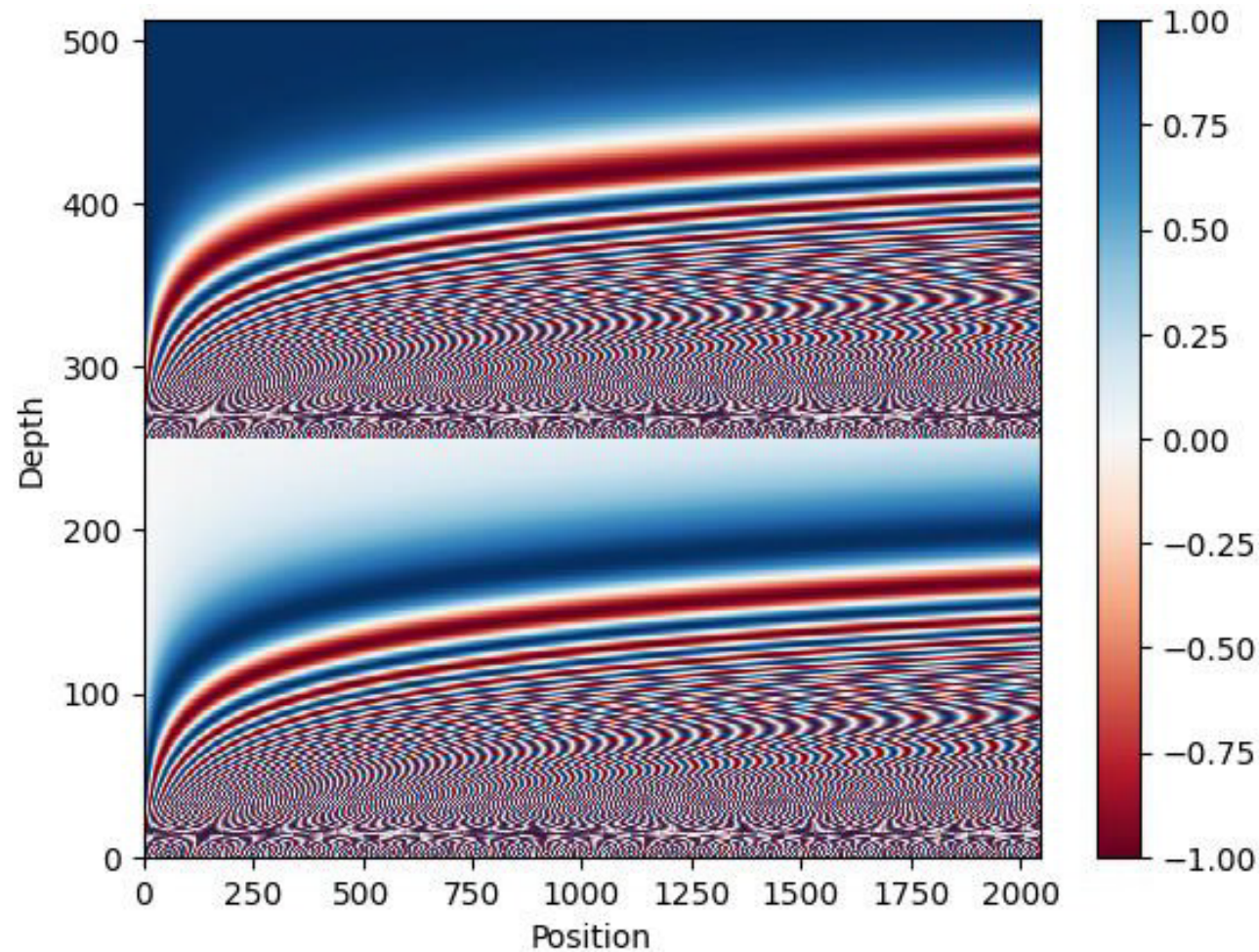
What is Positional Encoding?



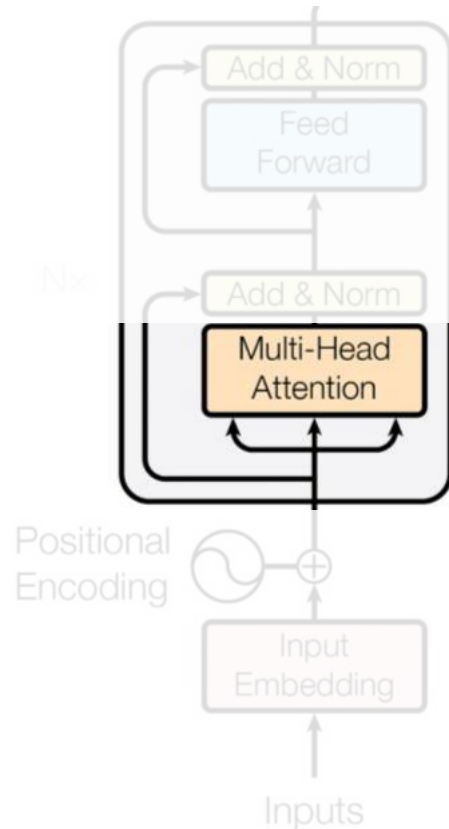
We only need to compute the positional encodings once and then reuse them for every sentence, no matter if it is training or inference.

Why Trigonometric Functions?

Trigonometric functions like **cos** and **sin** naturally represent a pattern that the model can recognize as continuous, so relative positions are easier to see for the model. By watching the plot of these functions, we can also see a regular pattern, so we can hypothesize that the model will see it too.



The Multi-Head Attention



Multi-Head Self Attention

- Self → The input sequence is used to create Q, K, V used in the attention mechanism
- Multi-Head → It's an attention mechanism split in "smaller" attention mechanisms that operate in parallel and whose partial results are then concatenated to obtain the final attention result
- Each head learns different attention patterns → the model captures diverse contextual information from different subspaces of the embeddings
- In other words: the embedding dimension, d_{model} , is split into multiple heads → each attention head focuses on different parts of the input sequence simultaneously to extract different patterns and contextual information

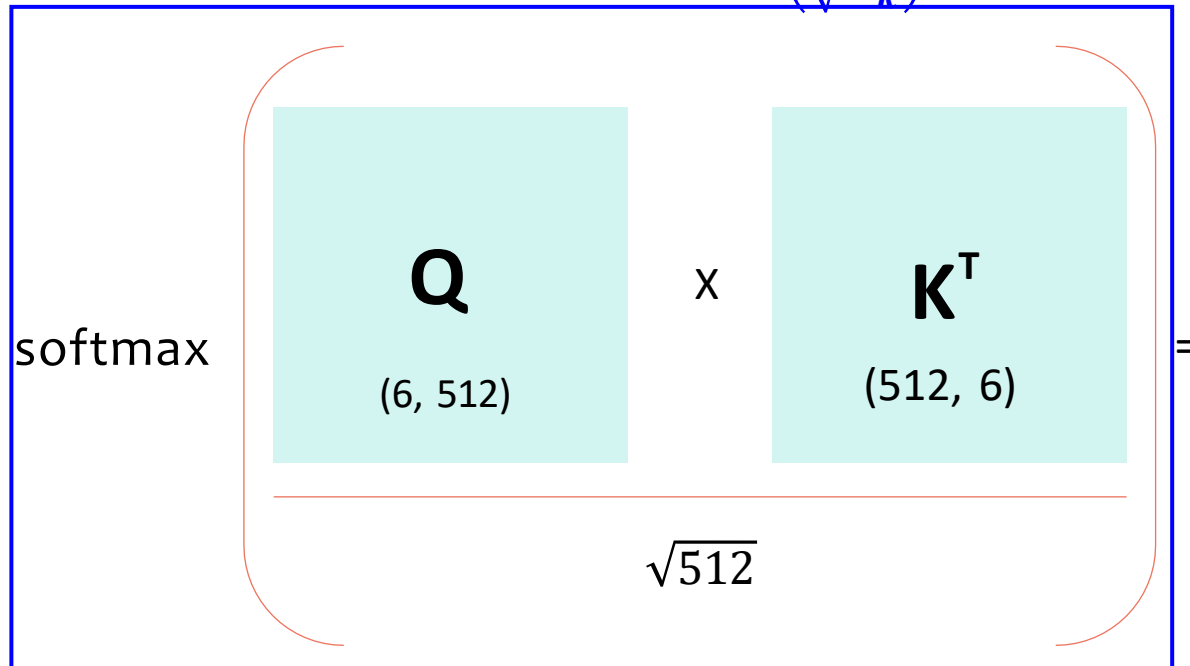
Self-Attention in Detail

Self-Attention allows the model to relate words to each other.

In this simple case we consider the sequence length $\text{seq} = 6$ and $\mathbf{d}_{\text{model}} = \mathbf{d}_k = 512$.

The matrices $\mathbf{Q}$, $\mathbf{K}$ and $\mathbf{V}$ are just the input sentence.

$$\text{Attention} = \text{softmax} \left(\frac{\mathbf{Q}\mathbf{K}^T}{\sqrt{d_k}} \right) \cdot \mathbf{V}$$



* for simplicity let's consider only one head, which makes $\mathbf{d}_{\text{model}} = \mathbf{d}_k$.

	YOUR	CAT	IS	A	LOVELY	CAT	Σ
YOUR	0.268	0.119	0.134	0.148	0.179	0.152	1
CAT	0.124	0.278	0.201	0.128	0.154	0.115	1
IS	0.147	0.132	0.262	0.097	0.218	0.145	1
A	0.210	0.128	0.206	0.212	0.119	0.125	1
LOVELY	0.146	0.158	0.152	0.143	0.227	0.174	1
CAT	0.195	0.114	0.203	0.103	0.157	0.229	1

(6, 6)

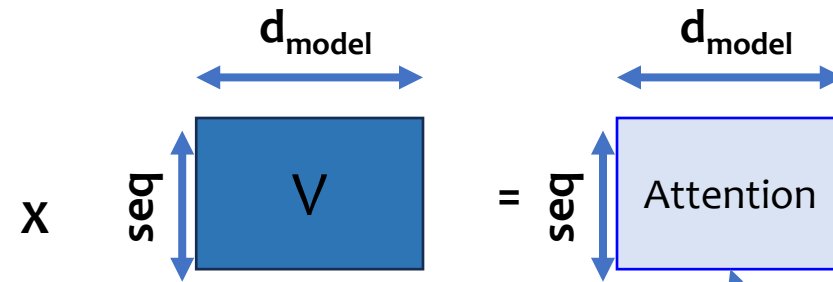
After softmax, sum of each row is equal to one

* all values are random (just as example).

Self-Attention in Detail

$$Attention = \text{softmax} \left(\frac{QK^T}{\sqrt{d_k}} \right) \cdot V$$

		seq						
								
		YOUR	CAT	IS	A	LOVELY	CAT	Σ
seq	YOUR	0.268	0.119	0.134	0.148	0.179	0.152	1
	CAT	0.124	0.278	0.201	0.128	0.154	0.115	1
	IS	0.147	0.132	0.262	0.097	0.218	0.145	1
	A	0.210	0.128	0.206	0.212	0.119	0.125	1
	LOVELY	0.146	0.158	0.152	0.143	0.227	0.174	1
	CAT	0.195	0.114	0.203	0.103	0.157	0.229	1



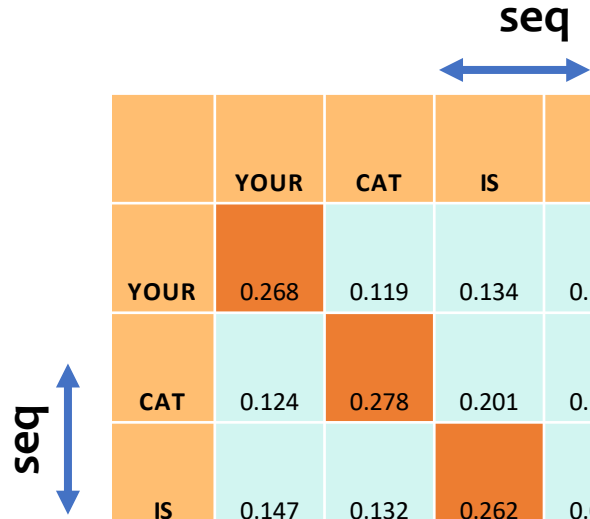
Each row in this matrix captures not only the meaning (given by the embedding) or the position in the sentence (represented by the positional encodings) but also each word's interaction with other words.

Row i of the matrix is word-vector i (in sequence S) mixed with attention scores of all word-vectors in the sequence

Self-Attention in Detail

Properties of Self-Attention

- Permutation invariant (if we do not consider the contribution of positional encoding)
- Requires no parameters. Up to now, the interaction between words has been driven by their embedding and the positional encodings. → This will change once we move to multi-head attention
- Largest values expected along the diagonal (interaction of word with itself)
- We can force some values to $-\infty$ to exclude some interactions → After applying the softmax these values will be '0' and the model will not learn those interactions → USED IN THE DECODER!



	YOUR	CAT	IS	A	LOVELY	CAT	Σ
YOUR	0.268	0.119	0.134	0.148	0.179	0.152	1
CAT	0.124	0.278	0.201	0.128	0.154	0.115	1
IS	0.147	0.132	0.262	0.097	0.218	0.145	1
A	0.210	0.128	0.206	0.212	0.119	0.125	1
LOVELY	0.146	0.158	0.152	0.143	0.227	0.174	1
CAT	0.195	0.114	0.203	0.103	0.157	0.229	1

Multi-Head Attention

$$\text{MultiHead}(Q, K, V) = \text{Concat}(\text{head}_1, \text{head}_1, \dots, \text{head}_h)W_O$$

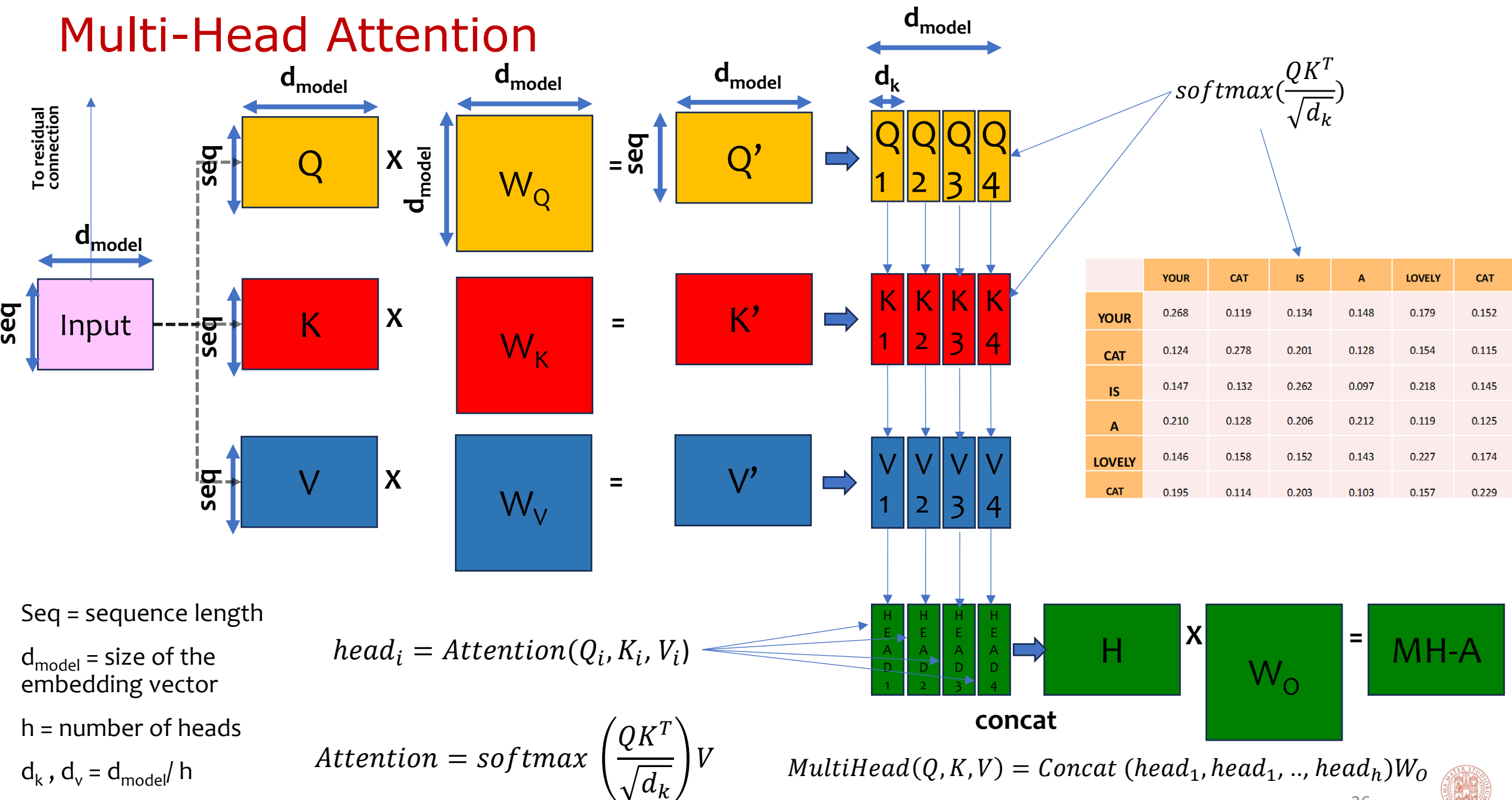
where

$$\text{head}_i = \text{Attention}(Q_i, K_i, V_i), \text{ for } i = 0 \dots h$$

where

$$\text{Attention} = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V$$

Multi-Head Attention

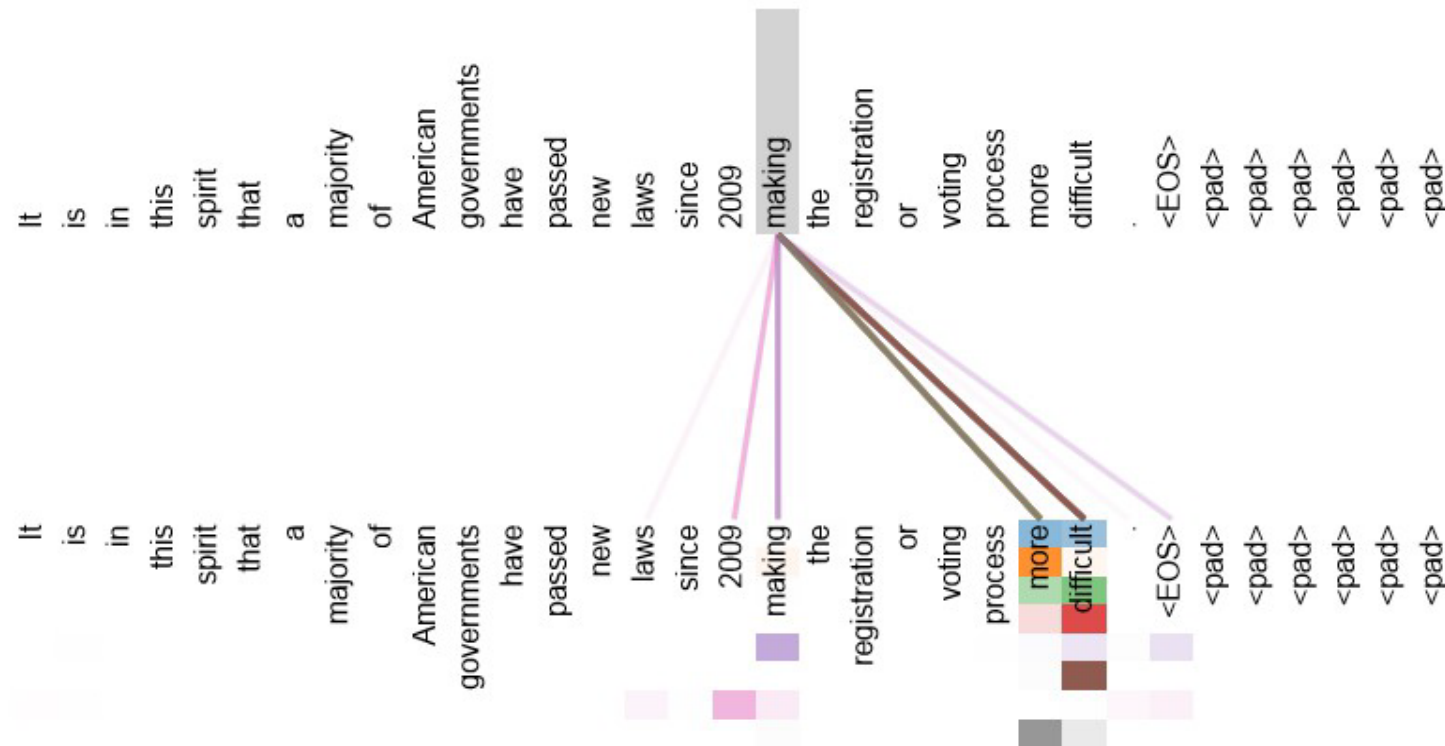


Attention Visualization

Why we want multi-head attention?

We want that each head looks at different aspects of the word (for example in some languages the same word can be seen as noun, verb and so on..)

Attention Visualizations



The Softmax Function

- Softmax is a mathematical function that converts a vector of values into a probability distribution, where each value is between 0 and 1 and the total sums to 1
- Softmax is applied to attention scores in Transformers to determine how much focus (or "attention") each word should receive relative to others in a sentence.

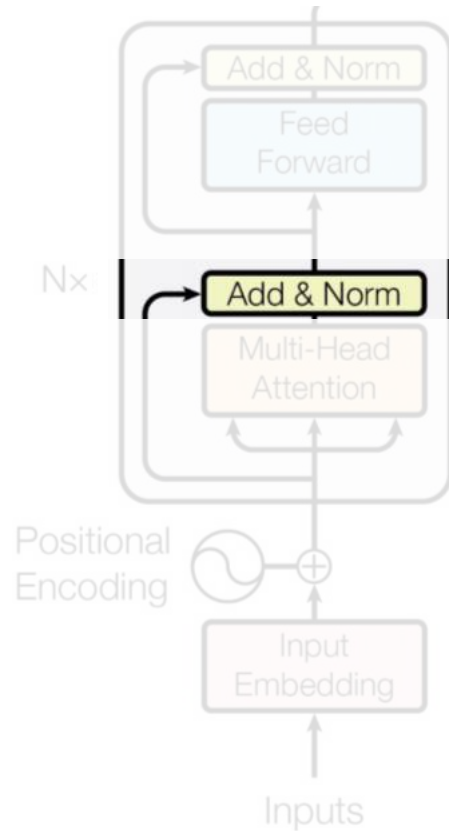
$$\text{Softmax}(\mathbf{z})_i = \frac{e^{z_i}}{\sum_{j=1}^N e^{z_j}}$$

$$\text{Softmax}(\mathbf{z})_i = \frac{e^{z_i - \max(\mathbf{z})}}{\sum_{j=1}^N e^{z_j - \max(\mathbf{z})}}$$

By hand now!

The Residual Connection and Normalization

Residual Connection

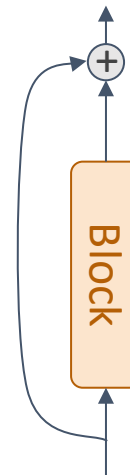


- Each module's output has the exact same shape as its input.
- Following ResNets, the module computes a "residual" instead of a new value:

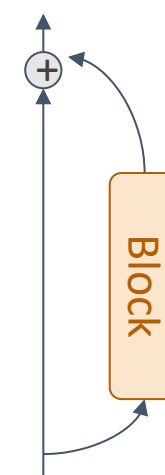
$$z_i = \text{Module}(x_i) + x_i$$

- This was shown to dramatically improve trainability.

"Skip connection"

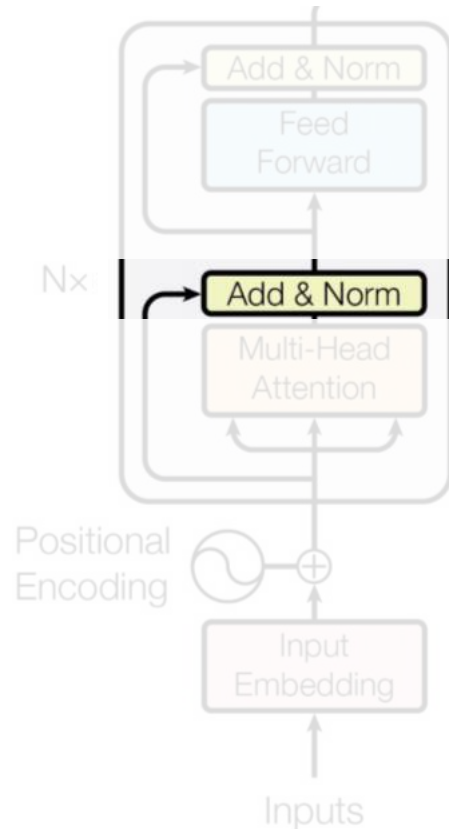


"Residual block"



The Residual Connection and Normalization

LayerNorm



- Layer Normalization also dramatically improves trainability.
- There are two different approaches for normalization
 - Original one → post-normalization
$$z_i = \text{LN}(\text{Module}(x_i) + x_i)$$
 - Modern one → pre-normalization
$$z_i = \text{Module}(\text{LN}(x_i)) + x_i$$

More on Layer Normalization

Batch of 3 items

ITEM 1

50.147
3314.825
...
...
8463.361
8.021

μ_1

σ_1^2

ITEM 2

1242.223
688.123
...
...
434.944
149.442

μ_2

σ_2^2

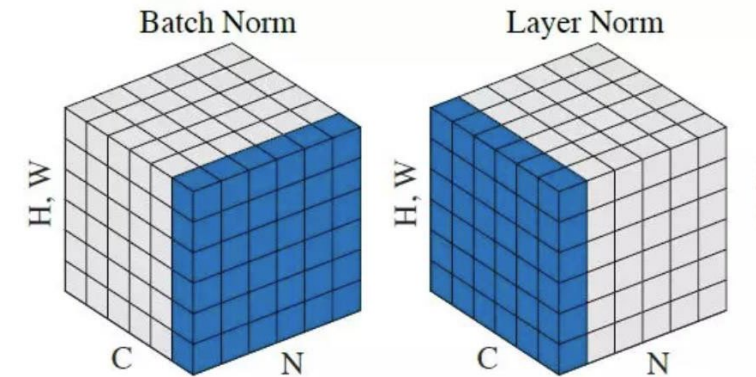
ITEM 3

9.370
4606.674
...
...
944.705
21189.444

μ_3

σ_3^2

$$\hat{x}_j = \frac{x_j - \mu_j}{\sqrt{\sigma_j^2 + \epsilon}}$$



We also introduce two parameters, usually called **gamma** (multiplicative) and **beta** (additive) that introduce some fluctuations in the data, because maybe having all values between 0 and 1 may be too restrictive for the network. The network will learn to tune these two parameters to introduce fluctuations when necessary.

The Feed Forward Layer

Point-wise MLP

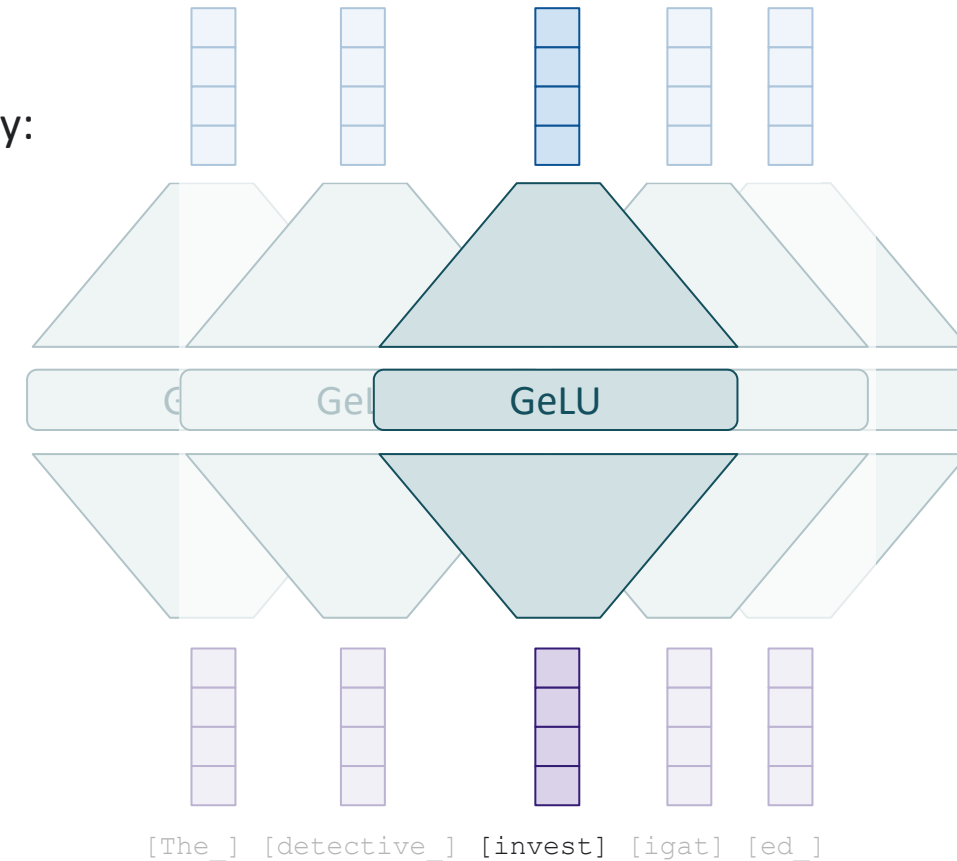
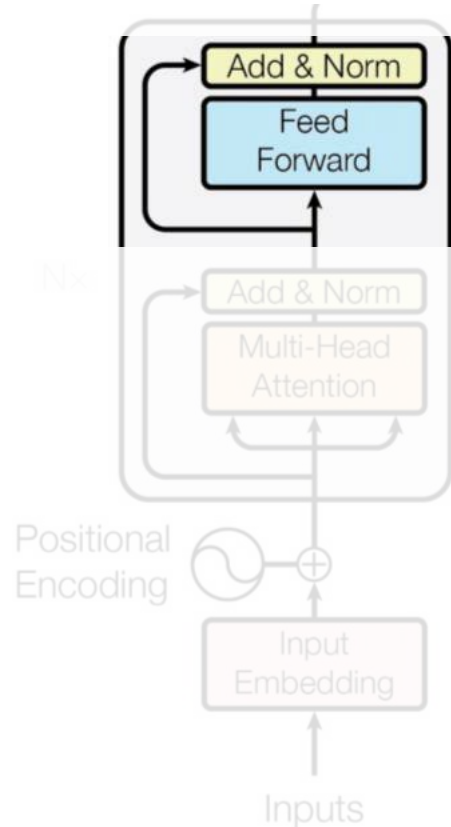
A simple MLP applied to each token individually:

$$z_i = W_2 \text{GeLU}(W_1 x + b_1) + b_2$$

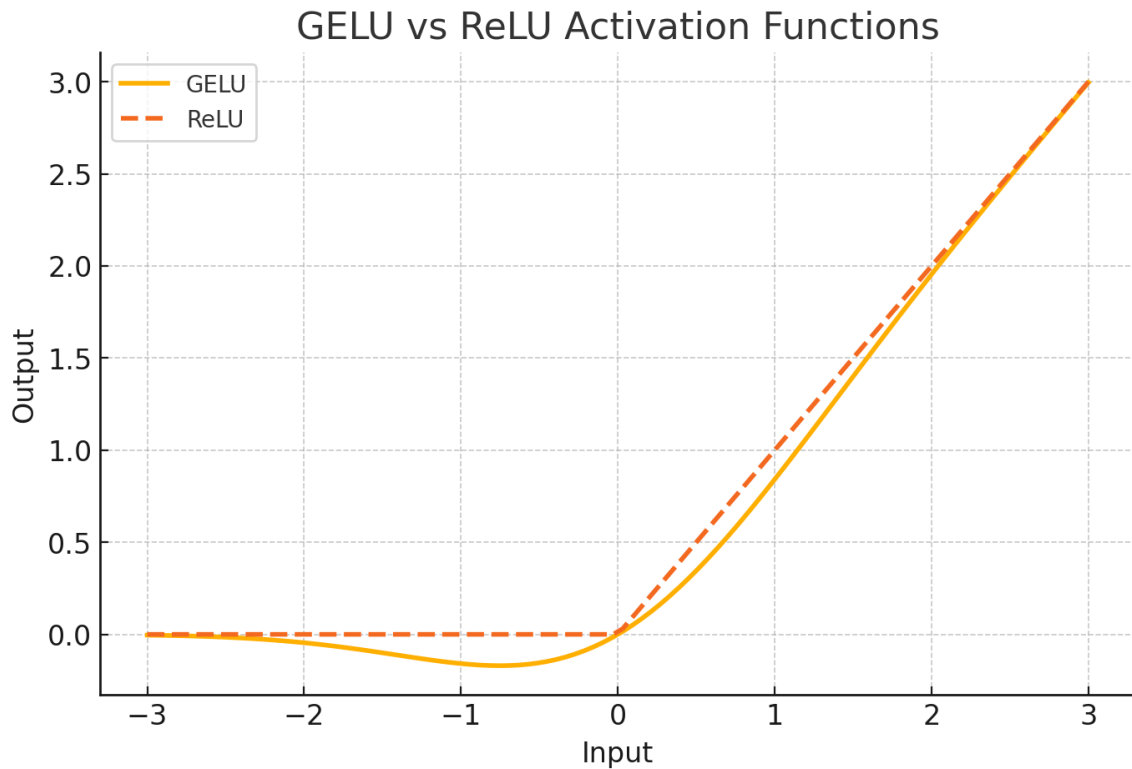
Think of it as each token pondering for itself about what it has observed previously. There's some weak evidence this is where "world knowledge" is stored, too.

It contains the bulk of the parameters. When people make giant models, this is what becomes giant.

Some people like to call it 1x1 convolution.

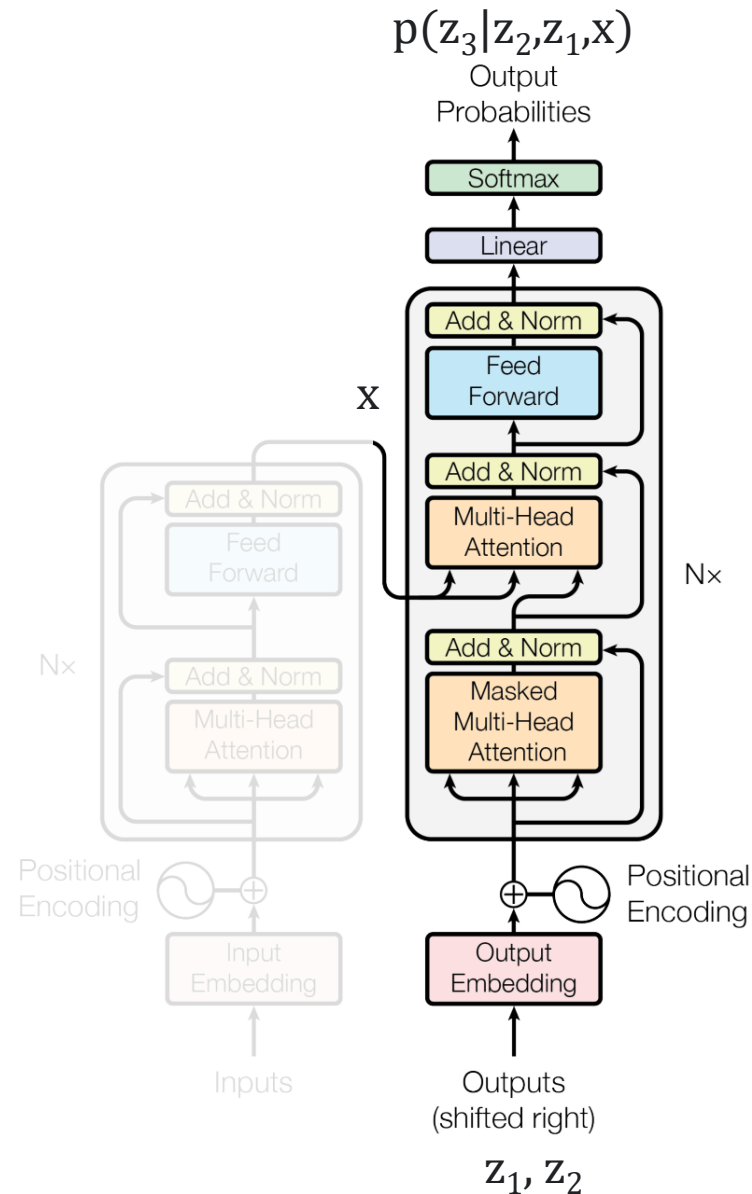


The GeLu Activation Function



- **Definition:** GELU activates neurons based on the probability that their input is positive.
- **Formula:**
$$\text{GELU}(x) = x\Phi(x) = x \cdot \frac{1}{2} \left[1 + \text{erf}(x/\sqrt{2}) \right],$$
 - Where $\Phi(x)$ is a gaussian distribution, usually represented as the error function (erf)
 - Usually GELU is approximated with tanh
$$0.5x \left(1 + \tanh \left[\sqrt{2/\pi} (x + 0.044715x^3) \right] \right)$$
- **Advantages:** Unlike ReLU, GELU keeps some small negatives and has a smoother gradient, aiding training stability.
- **NLP Use:** Common in transformers (e.g., BERT) for its flexibility and better performance.

The Decoder



Decoding / the Decoder (alternatively Generating / the Generator)

What we want to model: $p(z|x)$

for example, in translation: $p(z | \text{"the detective investigated"}) \forall z$

Seems impossible at first, but we can *exactly* decompose into tokens:

$$p(z|x) = p(z_1|x) p(z_2|z_1, x) p(z_3|z_2, z_1, x) \dots$$

Meaning, we can generate the answer one token at a time.

Each p is a full pass through the model.

For generating $p(z_3|z_2, z_1, x)$:

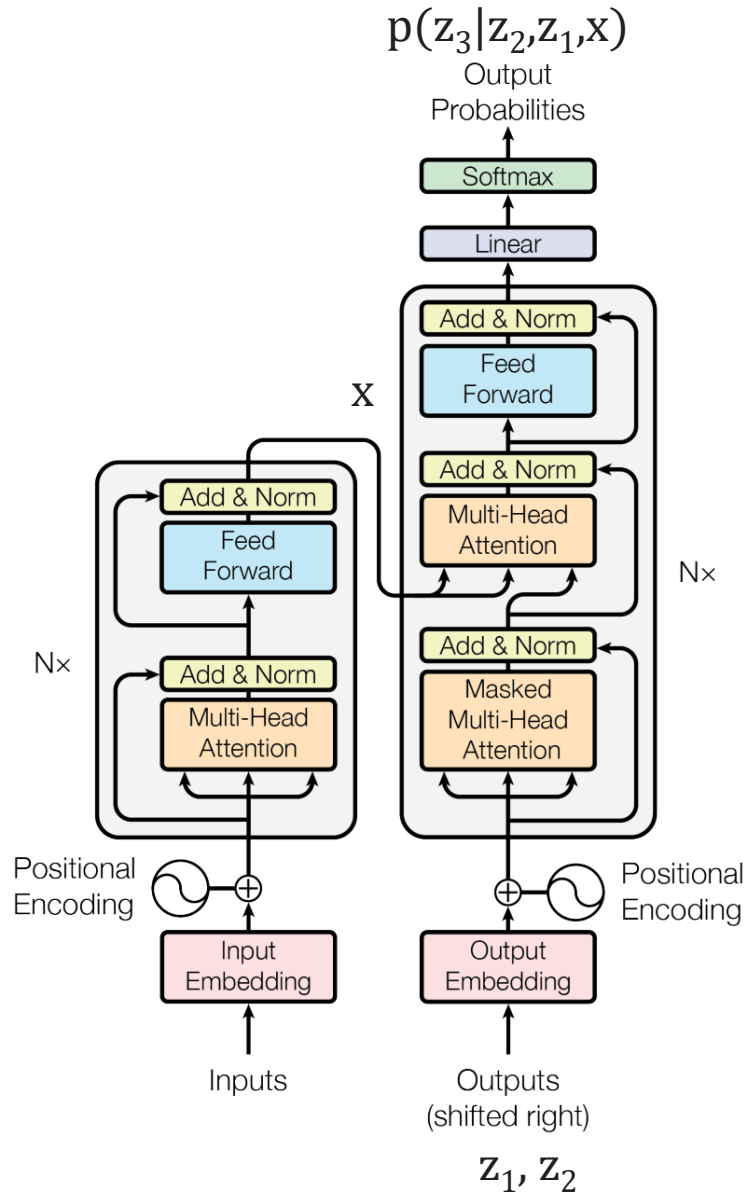
x comes from the encoder,

z_1, z_2 is what we have predicted so far, goes into the decoder.

Once we have $p(z|x)$ we still need to actually sample a sentence such as "le détective a enquêté". Many strategies: greedy, beam-search, ...

The Decoder

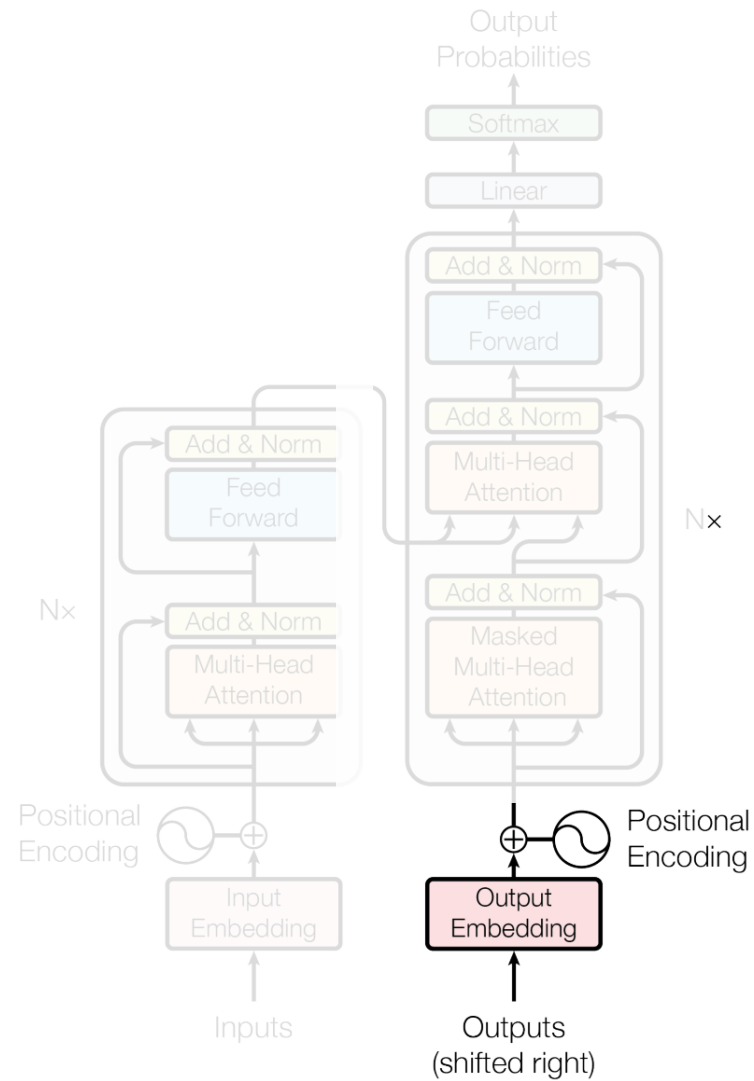
The first Multi-head attention is “masked”!



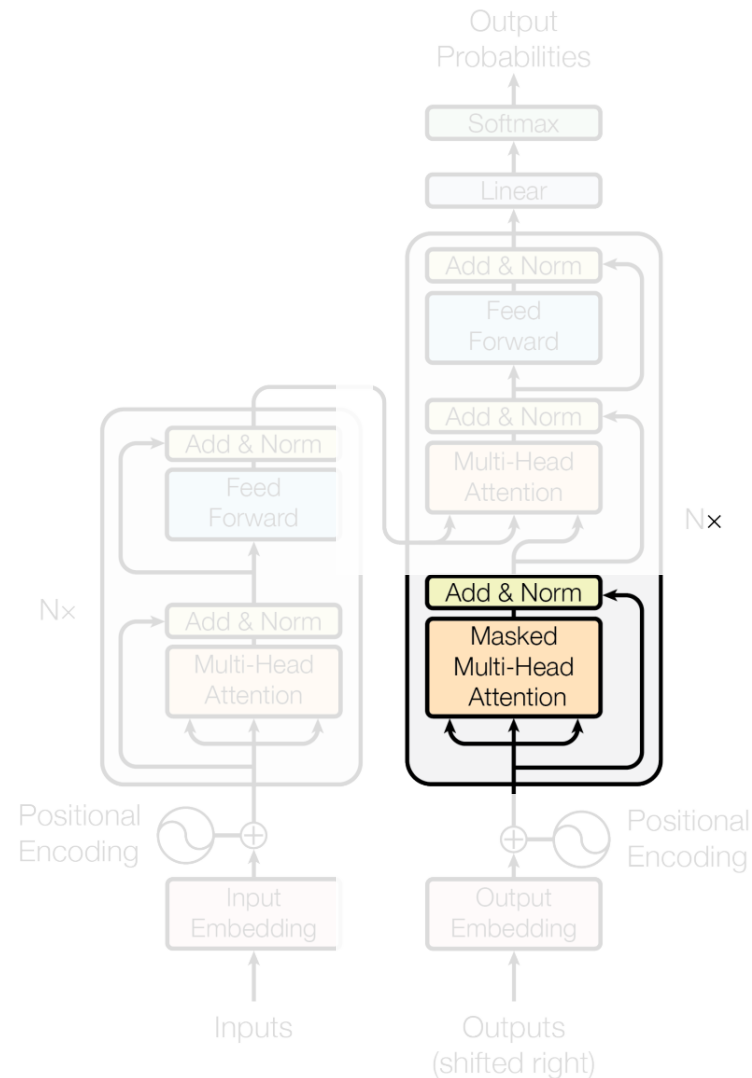
The MH-A in the decoder takes the K, V from ENCODER while the Q from the decoder flow

Output Embedding

Works exactly like the input embedding...



Masked Multi-Head Attention



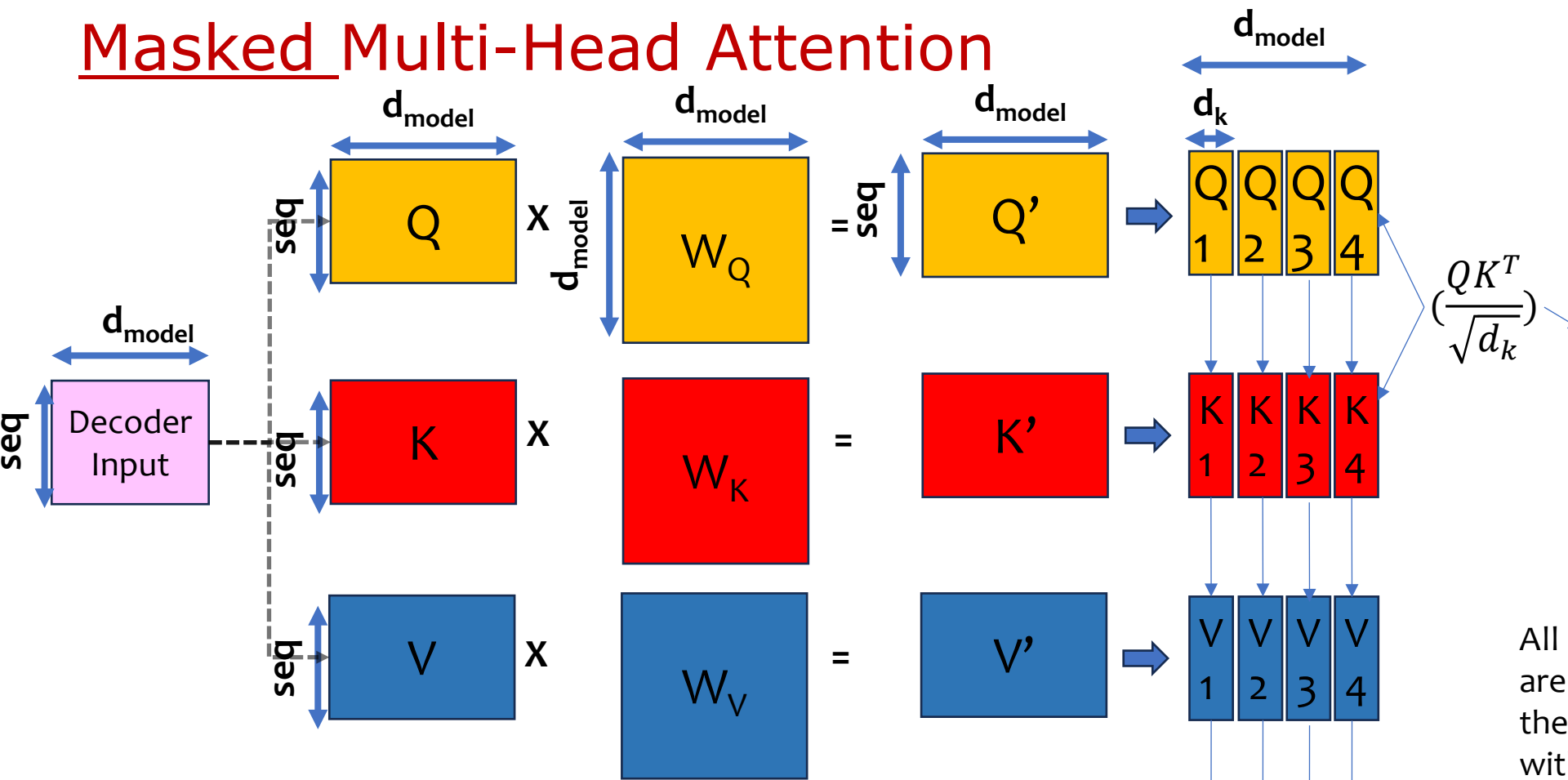
Our goal is to make the model causal

- It means the **output** at a certain position can only depend on the words on the previous positions.
- The model **must not** be able to see future words

	YOUR	CAT	IS	A	LOVELY	CAT
YOUR	0.268	0.119	0.134	0.148	0.179	0.152
CAT	0.124	0.278	0.201	0.128	0.154	0.115
IS	0.147	0.132	0.262	0.097	0.218	0.145
A	0.210	0.128	0.206	0.212	0.119	0.125
LOVELY	0.146	0.158	0.152	0.143	0.227	0.174
CAT	0.195	0.114	0.203	0.103	0.157	0.229

- How to do this in practice? We substitute values related to “future words (tokens)” with $-\infty$, which is then “o” after the softmax

Masked Multi-Head Attention



	YOUR	CAT	IS	A	LOVELY	CAT
YOUR	0.268	$-\infty$	$-\infty$	$-\infty$	$-\infty$	$-\infty$
CAT	0.124	0.278	$-\infty$	$-\infty$	$-\infty$	$-\infty$
IS	0.147	0.132	0.262	0.097	0.218	0.145
A	0.210	0.128	0.206	0.212	$-\infty$	$-\infty$
LOVELY	0.146	0.158	0.152	0.143	0.227	$-\infty$
CAT	0.195	0.114	0.203	0.103	0.157	0.229

All the values above the diagonal are replaced with $-\infty$ before applying the softmax, which will replace them with zero.

Seq = sequence length

d_{model} = size of the embedding vector

h = number of heads

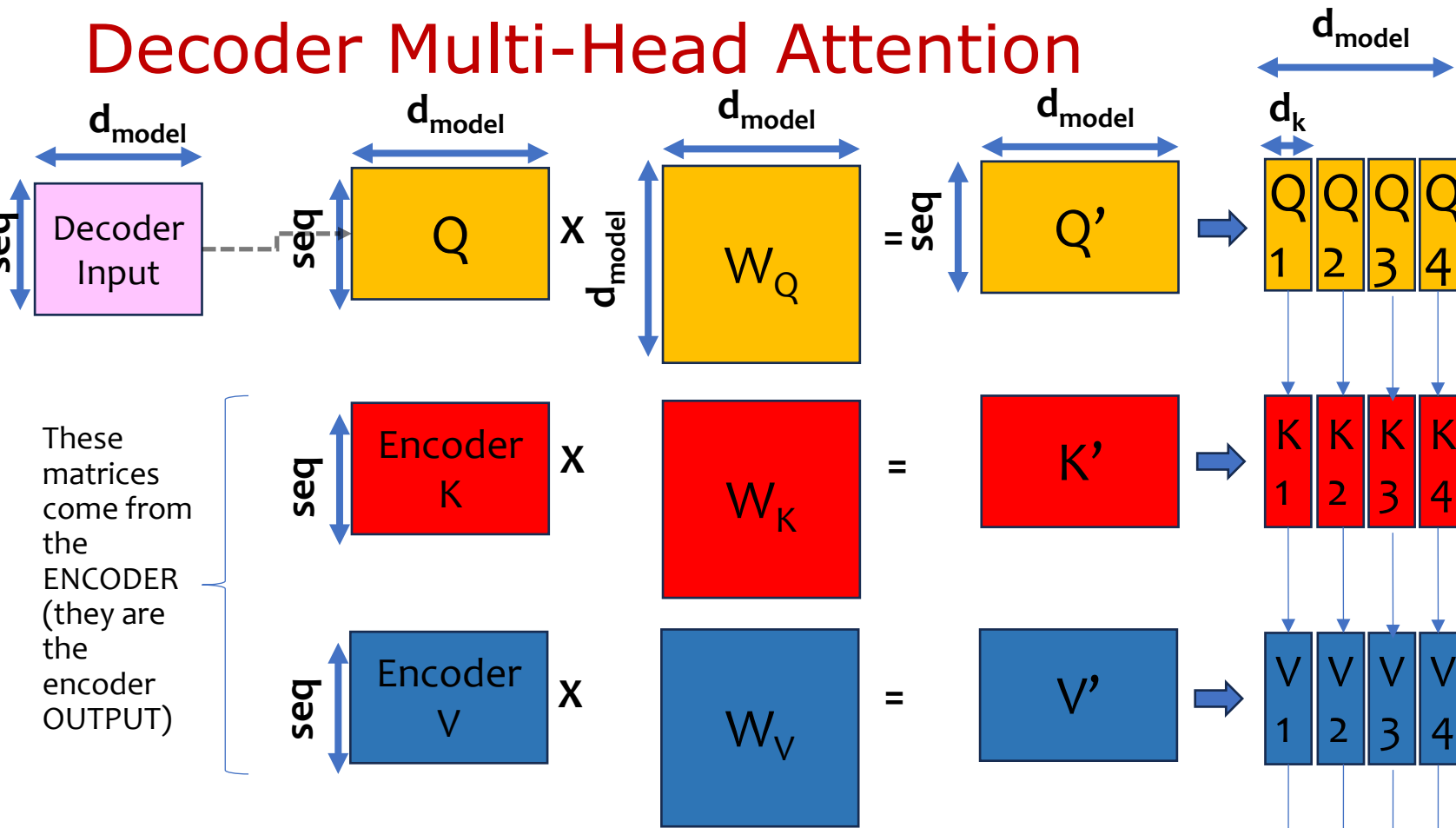
$d_k, d_v = d_{\text{model}} / h$

$$\text{head}_i = \text{Attention}(Q_i, K_i, V_i)$$

$$\text{Attention} = \text{softmax} \left(\frac{QK^T}{\sqrt{d_k}} \right) V$$

$$\text{MultiHead}(Q, K, V) = \text{Concat}(\text{head}_1, \text{head}_1, \dots, \text{head}_h) W_O$$

Decoder Multi-Head Attention



These matrices come from the ENCODER (they are the encoder OUTPUT)

Seq = sequence length

d_{model} = size of the embedding vector

h = number of heads

$d_k, d_v = d_{\text{model}} / h$

$$\text{head}_i = \text{Attention}(Q_i, K_i, V_i)$$

$$\text{Attention} = \text{softmax} \left(\frac{QK^T}{\sqrt{d_k}} \right) V$$

$$\text{MultiHead}(Q, K, V) = \text{Concat}(\text{head}_1, \text{head}_1, \dots, \text{head}_h) W_O$$

Inference and Training of a Transformer Model

By hand now!

TRANSFORMERS VISUALIZATION

<https://poloclub.github.io/transformer-explainer/>

LLM VISUALIZATION

<https://bbycroft.net/llm>

Training



I love you very much



Ti amo molto

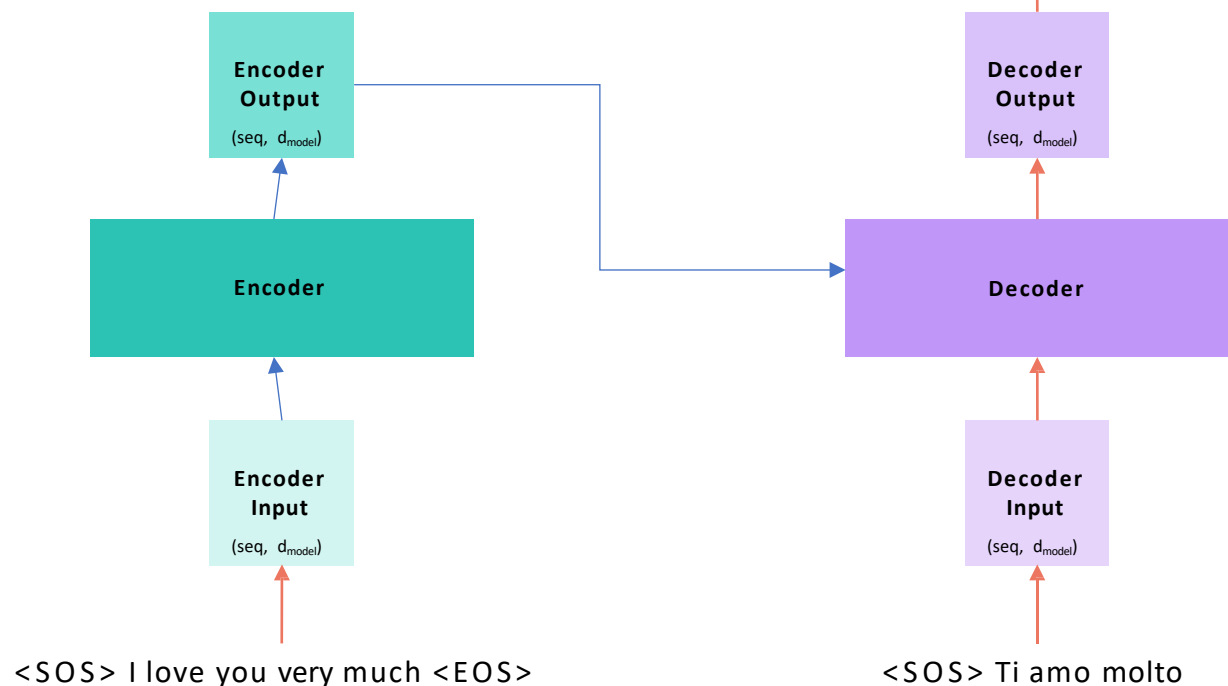


Training

Time Step = 1

It all happens in one time step!

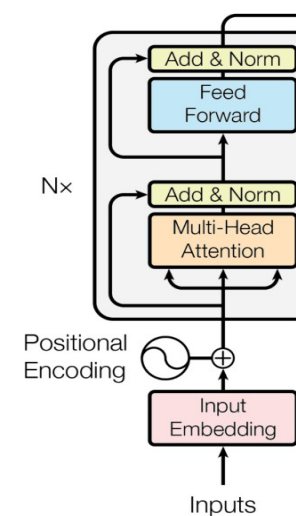
The encoder outputs, for each word a vector that not only captures its meaning (the embedding) or the position, but also its interaction with other words by means of the multi-head attention.



Ti amo molto <EOS>

* This is called the "label" or the "target"

Cross Entropy Loss



We prepend the <SOS> token at the beginning. That's why the paper says that the decoder input is shifted right.

Inference



I love you very much

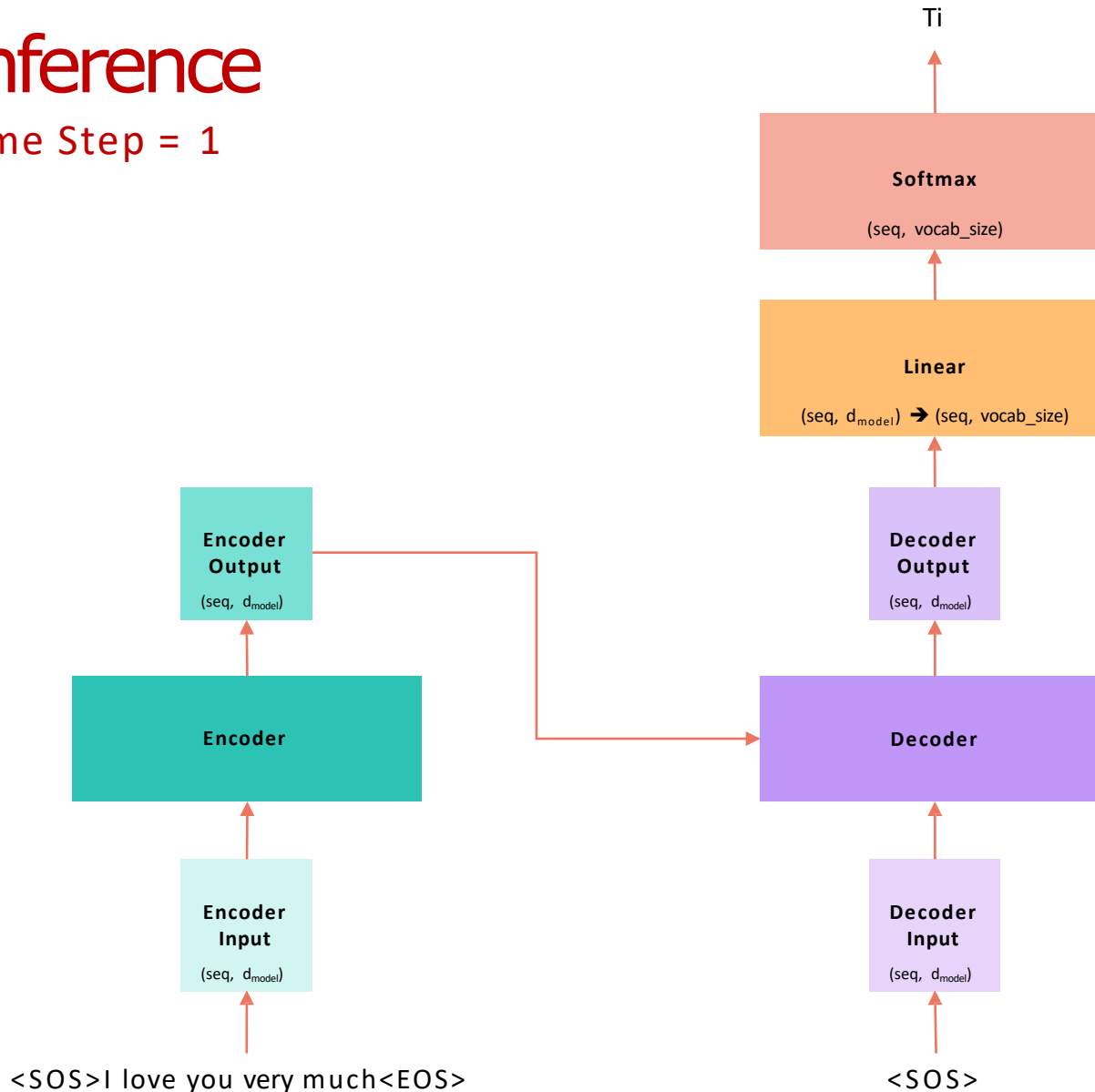


Ti amo molto



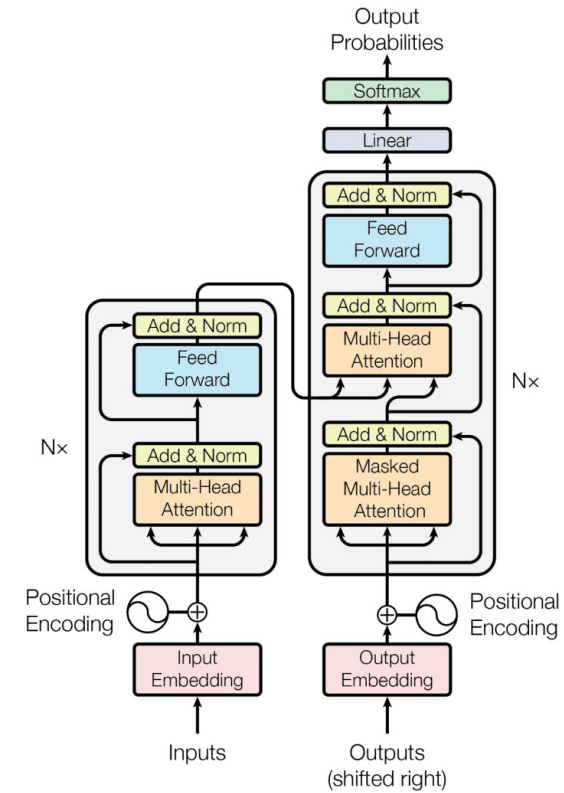
Inference

Time Step = 1



We select a token from the vocabulary corresponding to the position of the token with the maximum value.

The output of the last layer is commonly known as **logits**



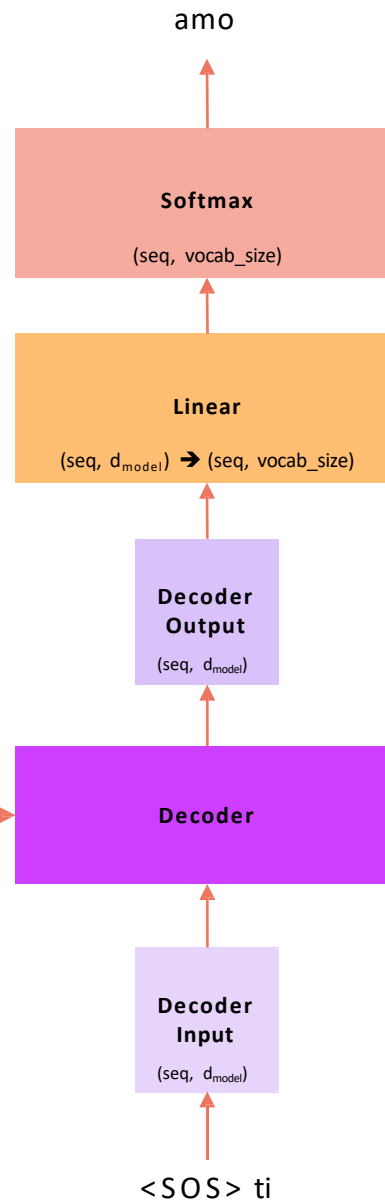
* Both sequences will have same length thanks to padding

Inference

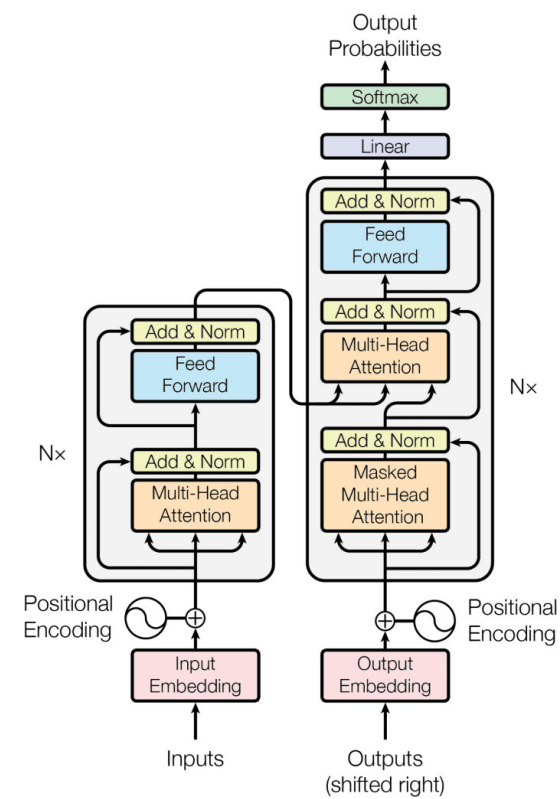
Time Step = 2

Use the encoder output from the first time step

<SOS>I love you very much<EOS>



Since decoder input now contains **two** tokens, we select the softmax corresponding to the second token.



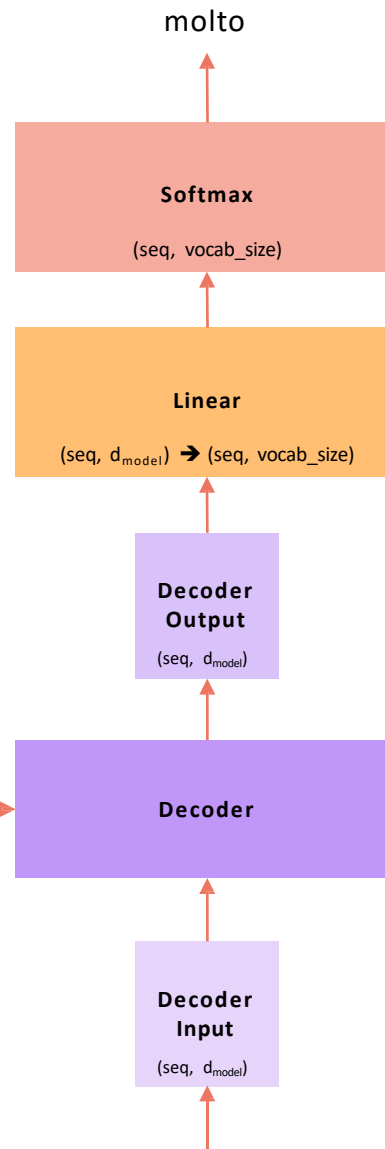
Append the previously output word to the decoder input

Inference

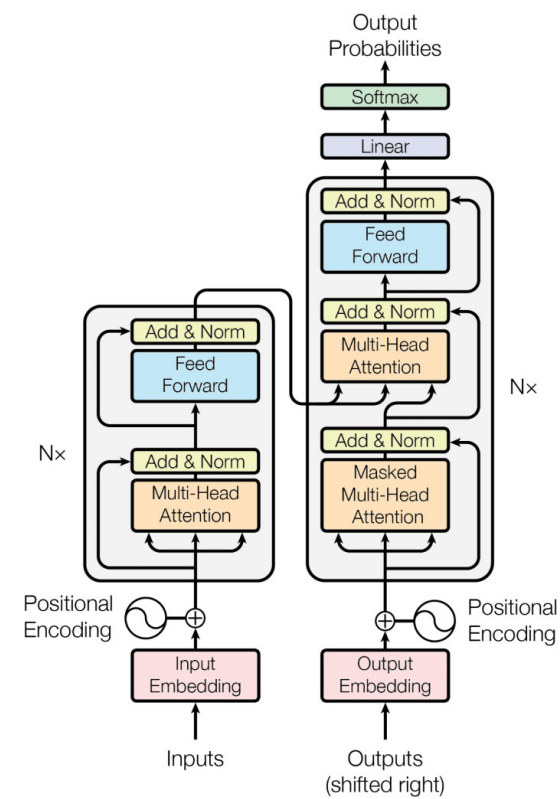
Time Step = 3

Use the encoder output from the first time step

<SOS>I love you very much<EOS>



Since decoder input now contains **three** tokens, we select the softmax corresponding to the third token.



Append the previously output word to the decoder input

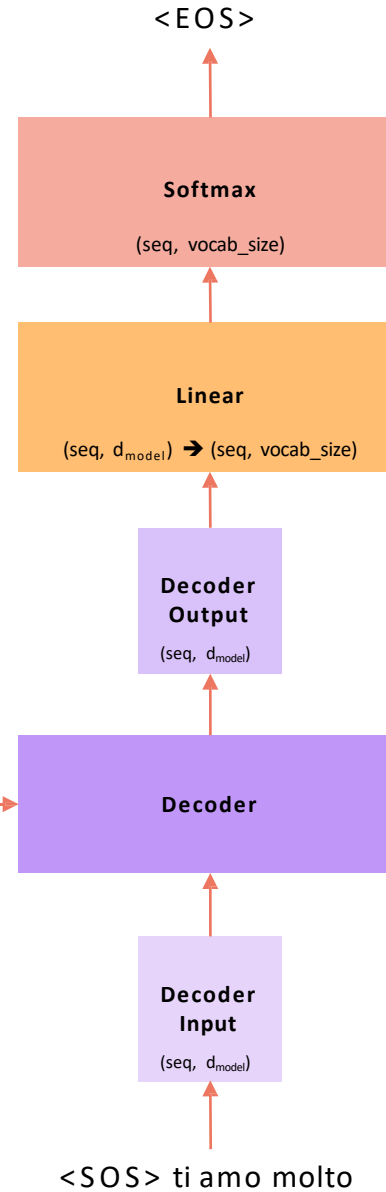


Inference

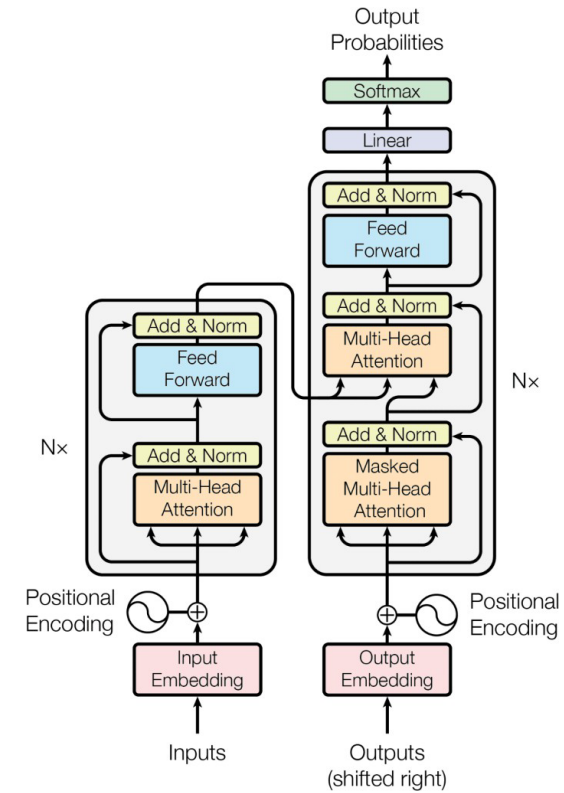
Time Step = 4

Use the encoder output from the first time step

<SOS>I love you very much<EOS>



Since decoder input now contains **four** tokens, we select the softmax corresponding to the fourth token.



Append the previously output word to the decoder input



Inference strategy

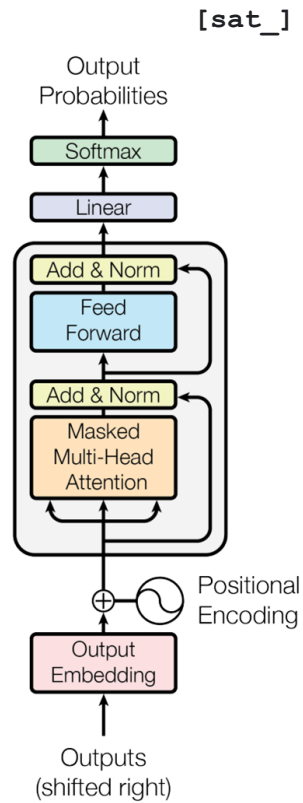
- We selected, at every step, the word with the maximum softmax value. This strategy is called **greedy** and usually does not perform very well.
- A better strategy is to select at each step the top B words and evaluate all the possible next words for each of them and at each step, keeping the top B most probable sequences. This is the **Beam Search** strategy and generally performs better.



Applications

Decoder-only

GPT

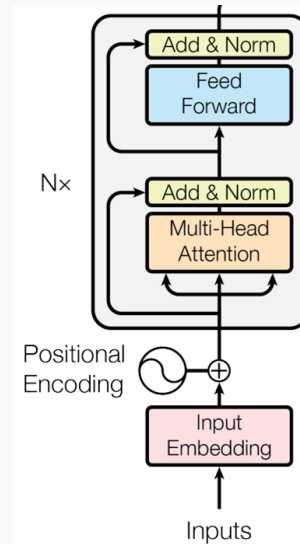


[START] [The_] [cat_] [sat_] [the_] [mat_] [MASK]

Encoder-only

BERT

[*] [*] [sat_] [*] [the_] [*]



[The_] [cat_] [MASK] [on_] [MASK] [mat_] [MASK]

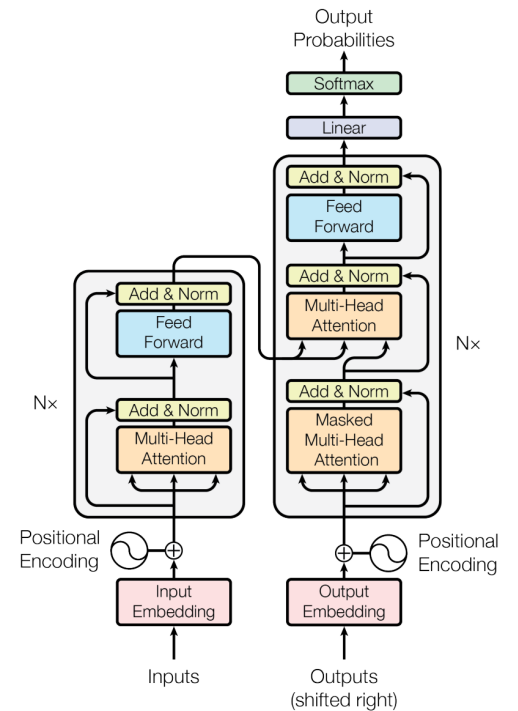
Enc-Dec

T5

Das ist gut.

A storm in Attala caused 6 victims.

This is not toxic.



Translate EN-DE: This is good.

Summarize: state authorities dispatched...

Is this toxic: You look beautiful today!

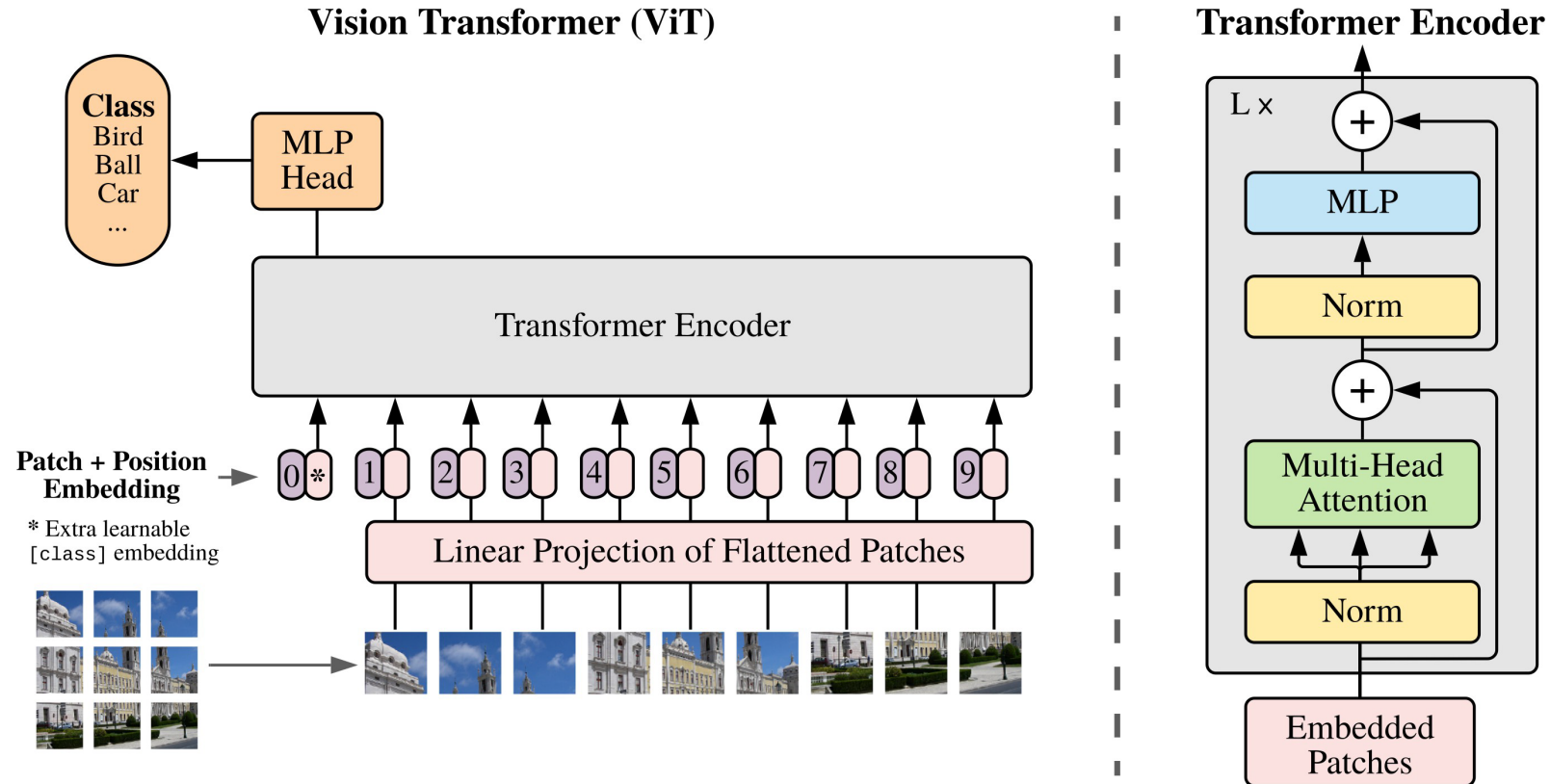


Language Translation Application

Table 2: The Transformer achieves better BLEU scores than previous state-of-the-art models on the English-to-German and English-to-French newstest2014 tests at a fraction of the training cost.

Model	BLEU		Training Cost (FLOPs)	
	EN-DE	EN-FR	EN-DE	EN-FR
ByteNet [18]	23.75			
Deep-Att + PosUnk [39]		39.2		$1.0 \cdot 10^{20}$
GNMT + RL [38]	24.6	39.92	$2.3 \cdot 10^{19}$	$1.4 \cdot 10^{20}$
ConvS2S [9]	25.16	40.46	$9.6 \cdot 10^{18}$	$1.5 \cdot 10^{20}$
MoE [32]	26.03	40.56	$2.0 \cdot 10^{19}$	$1.2 \cdot 10^{20}$
Deep-Att + PosUnk Ensemble [39]		40.4		$8.0 \cdot 10^{20}$
GNMT + RL Ensemble [38]	26.30	41.16	$1.8 \cdot 10^{20}$	$1.1 \cdot 10^{21}$
ConvS2S Ensemble [9]	26.36	41.29	$7.7 \cdot 10^{19}$	$1.2 \cdot 10^{21}$
Transformer (base model)	27.3	38.1	$3.3 \cdot 10^{18}$	
Transformer (big)	28.4	41.8	$2.3 \cdot 10^{19}$	

Image Transformer — ViT



- Split an image into fixed-size patches, linearly embed each of them, add position embeddings, and feed the resulting sequence of vectors to a standard Transformer encoder.

An Image is Worth 16x16 Words: Transformers for Image Recognition at Scale [Dosovitskiy *et al.*, 2020]

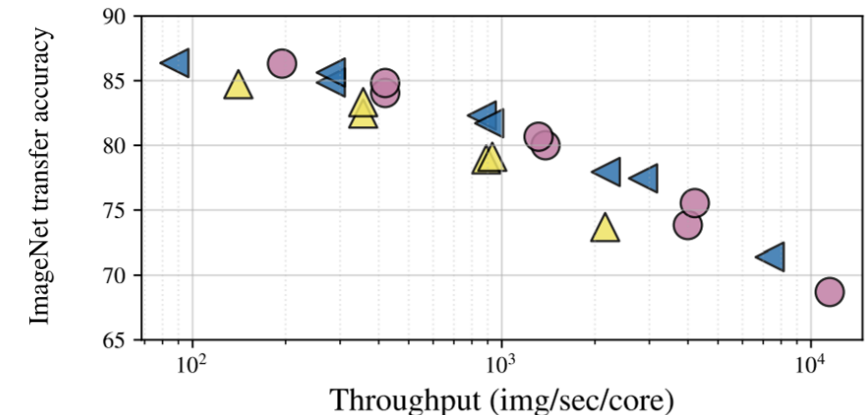
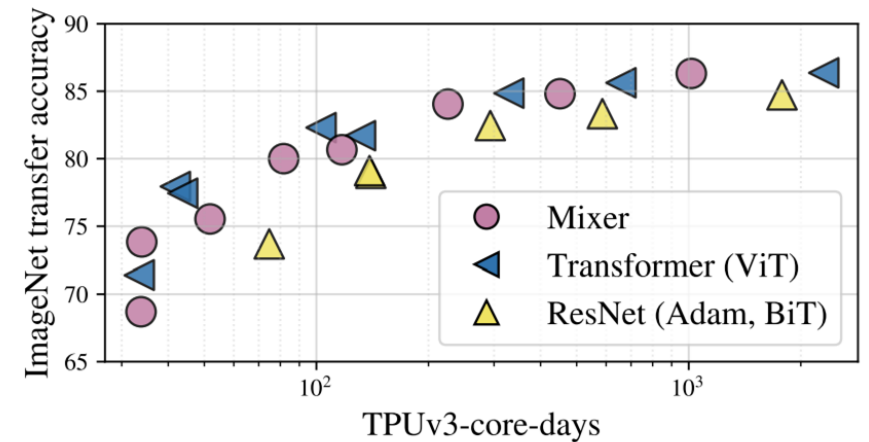
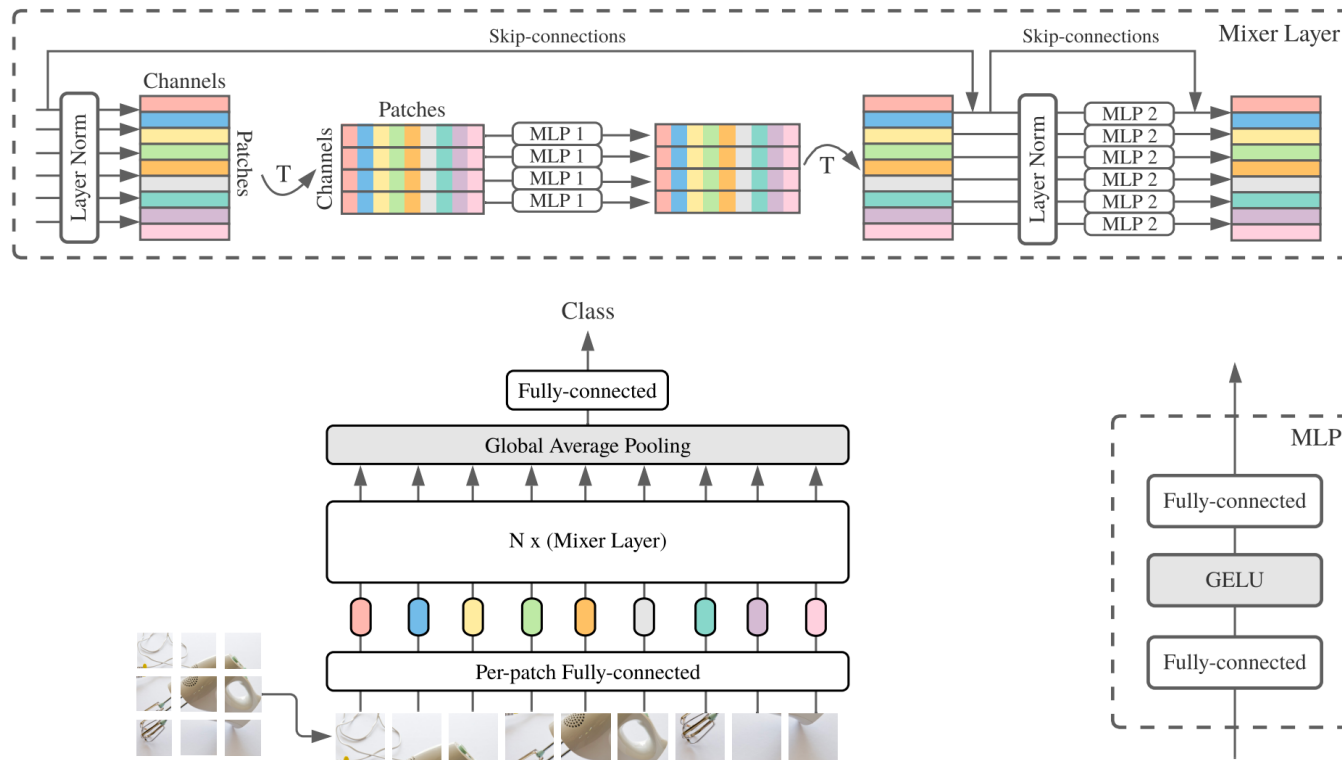
MLP-Mixer

After ViT answered the question "Are convolutions really needed to process images?" with NO...

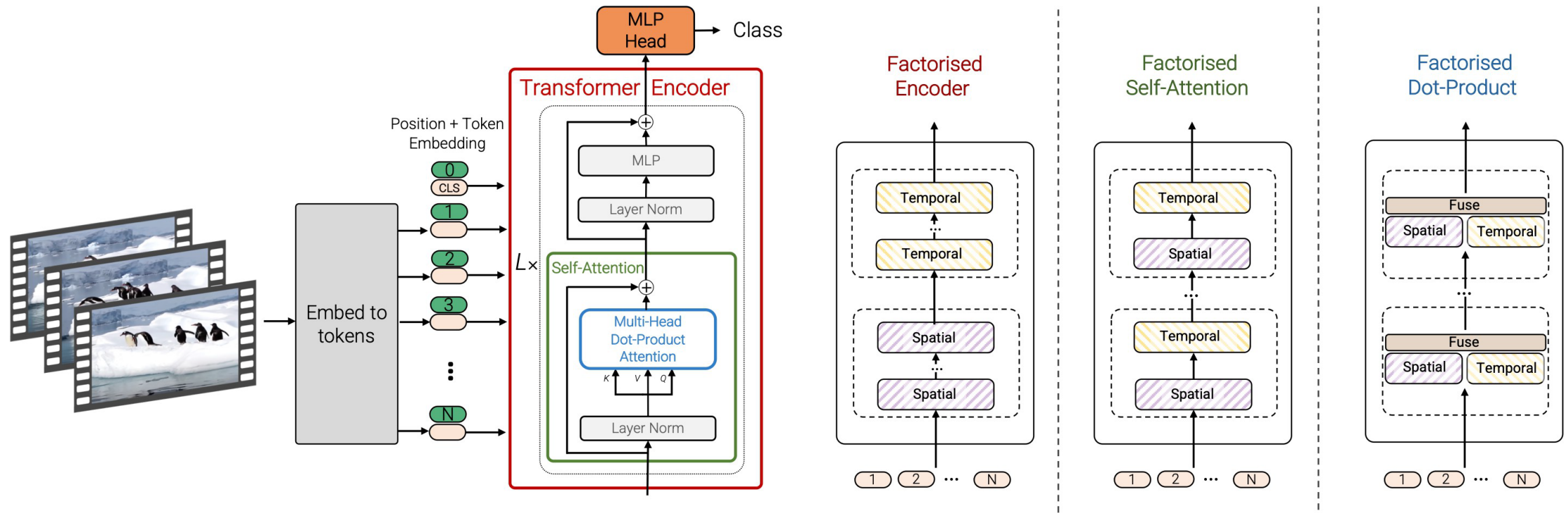
We wondered if self-attention is really needed?

The role of self-attention is to "mix" information across tokens.

Another simple way to achieve this, is to "transpose" tokens and run that through an MLP:

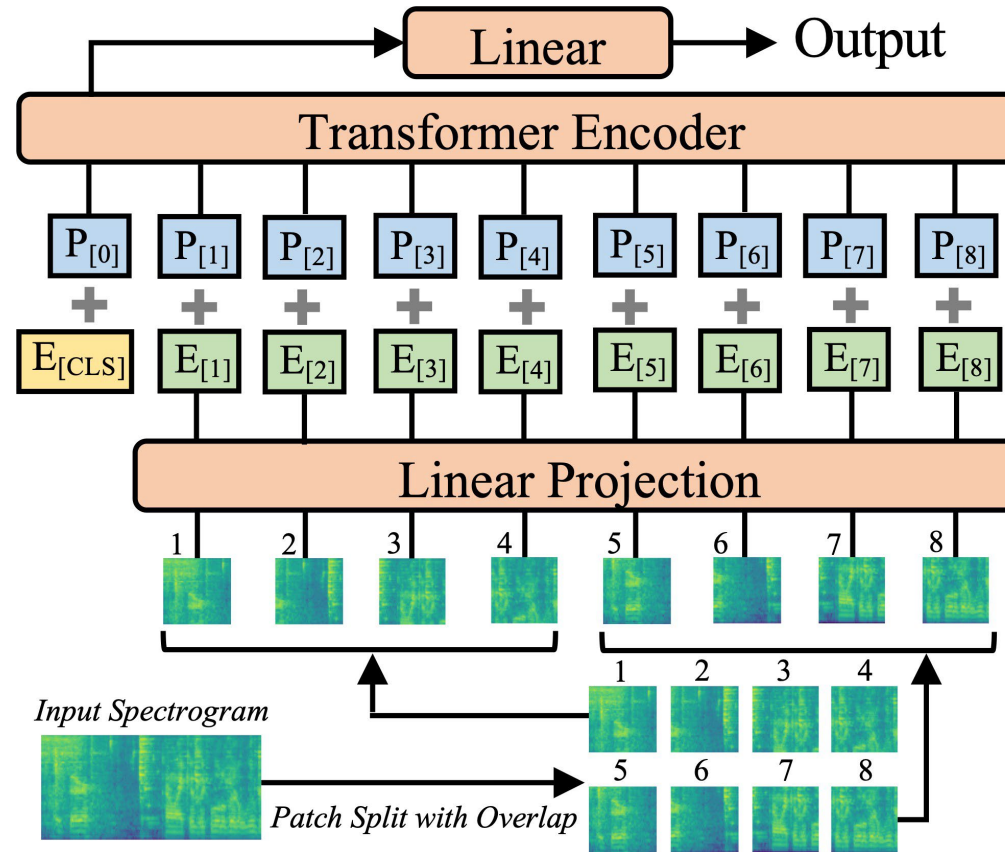


Video Transformer — ViViT



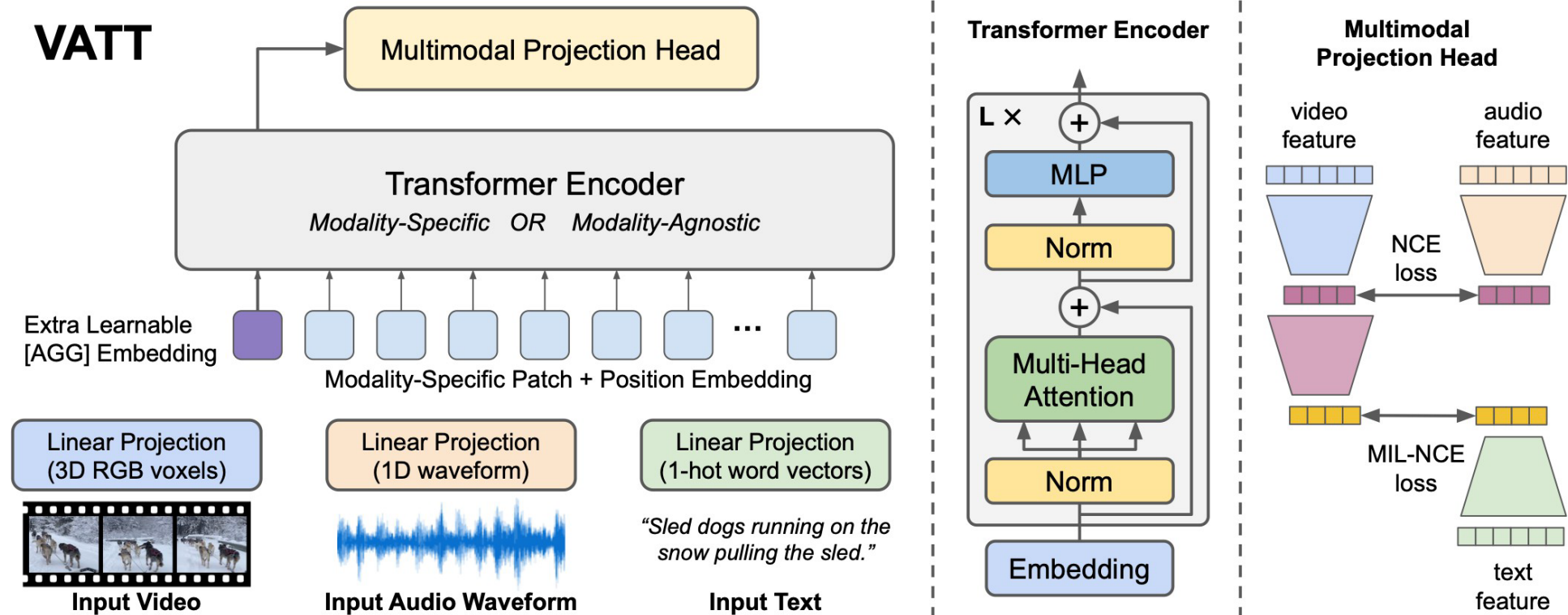
- Extract spatiotemporal tokens from the video, then encoded by a series of transformer layers.
- Factorize the spatial- and temporal-dimensions of the input to handle the long sequences.

Audio Transformer — AST



- Split the 2D audio spectrogram a sequence of 16×16 patches with overlap, and then linearly projected to a sequence of 1-D patch embeddings.

Multi-Modal Transformer — VATT



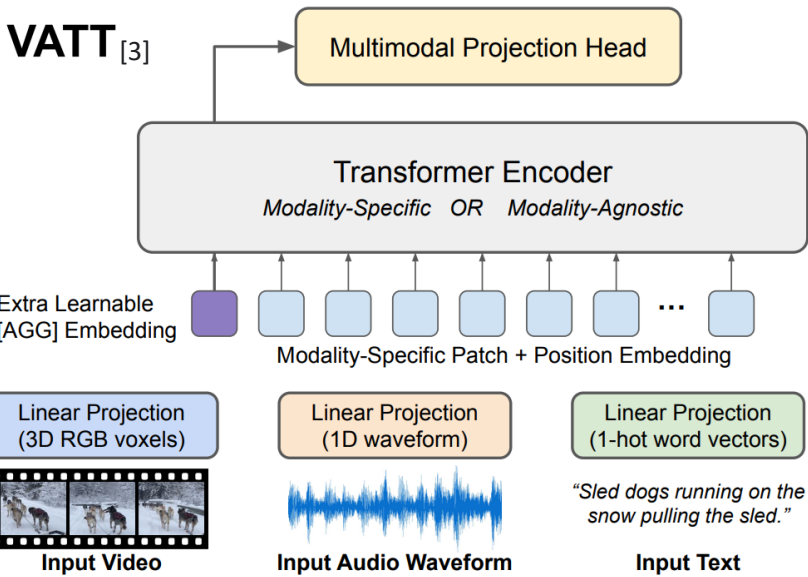
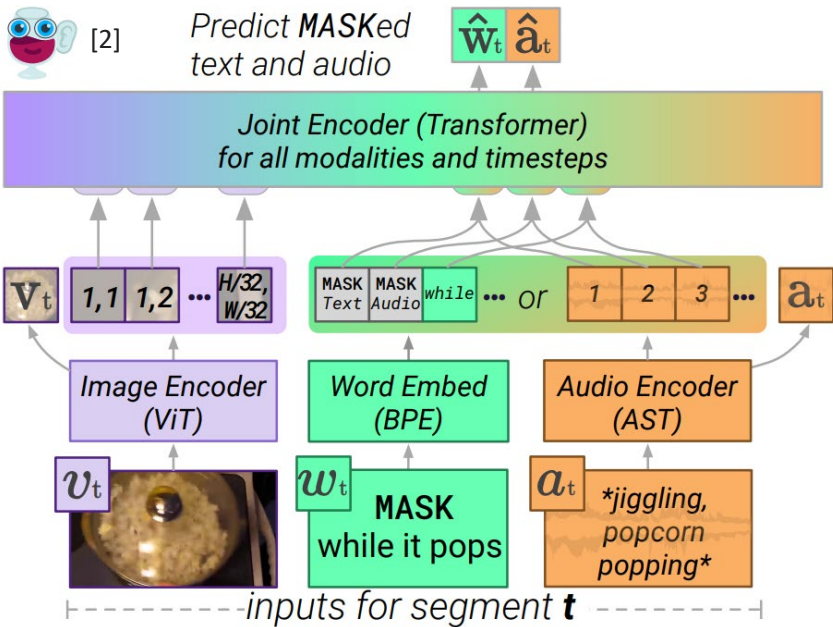
- Linearly project each modality into a feature vector and feed it into a Transformer encoder.

Anything you can tokenize, you can feed to Transformer

ca 2021 and onwards

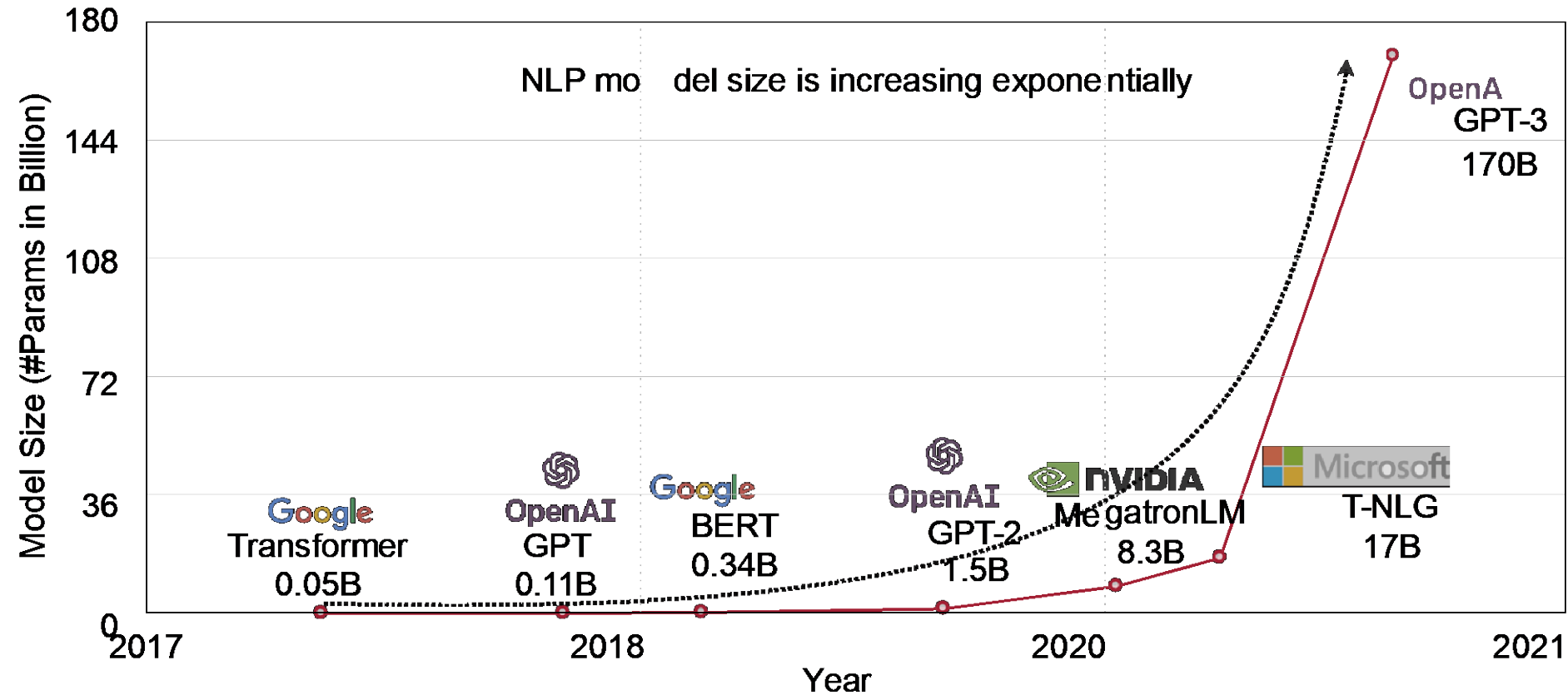
Tokenize different modalities each in their own way (some kind of "patching"), and send them all jointly into a Transformer...

Seems to just work...
Currently an explosion of works doing this!



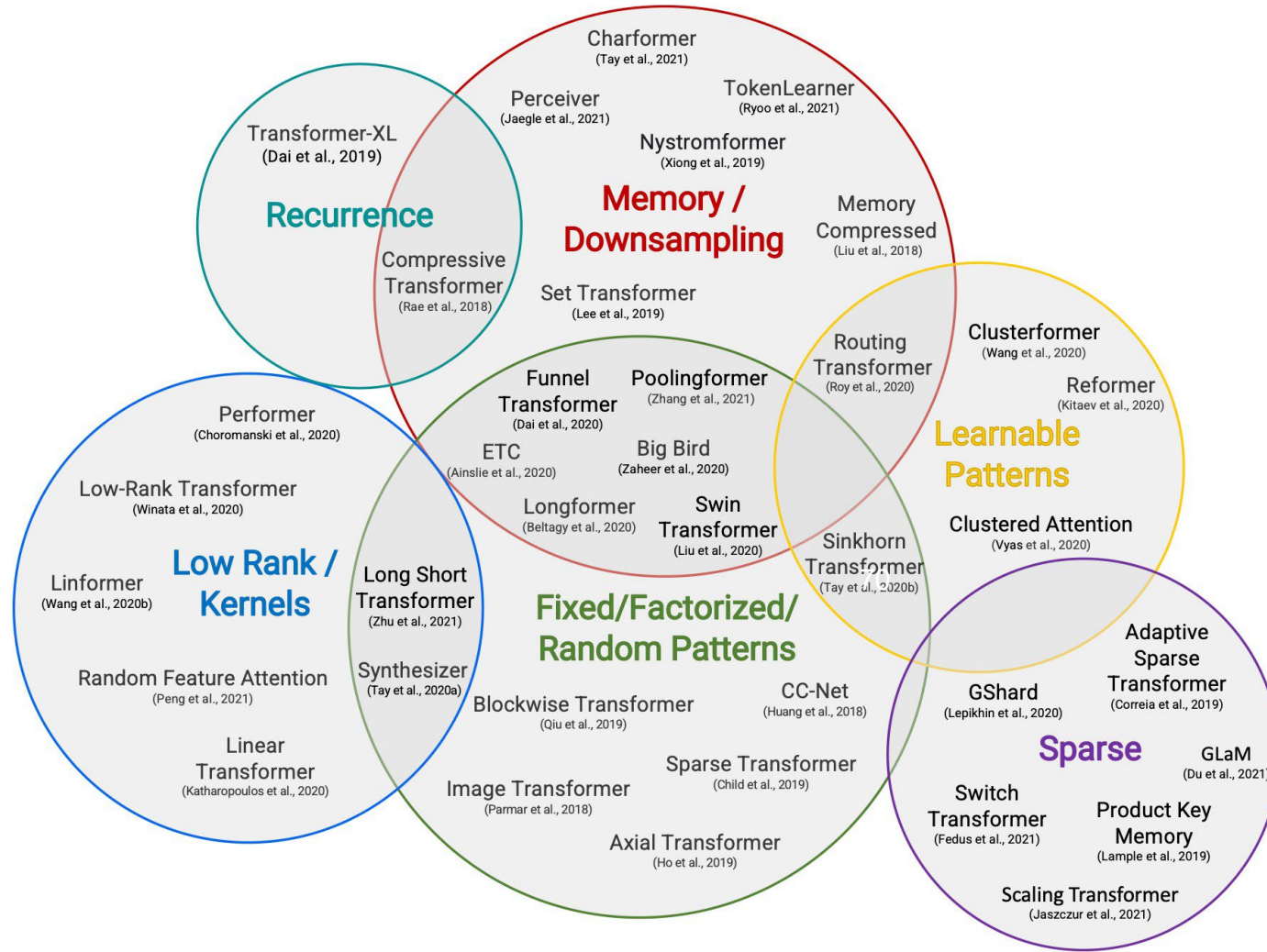
Images from:
[1] LIMoE by B Mustafa, C Riquelme, J Puigcerver, R Jenatton, N Houlsby
[2] MERLOT Reserve by R Zellers, J Lu, X Lu, Y Yu, Y Zhao, M Salehi, A Kusupati, J Hessel, A Farhadi, Y Choi
[3] VATT by H Akbari, L Yuan, R Qian, W-H Chuang, S-F Chang, Y Cui, B Gong

Transformers are Big!



Efficient Transformers Architectures

Efficient Transformers

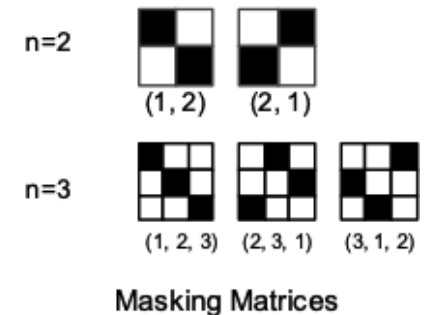
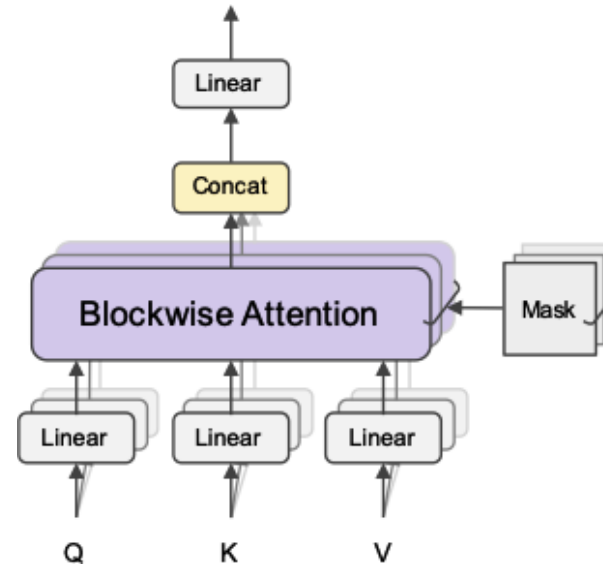


Efficient Transformers: A Survey [Tay et al., 2020]

Sparse Attention — BlockBERT

Blockwise Attention

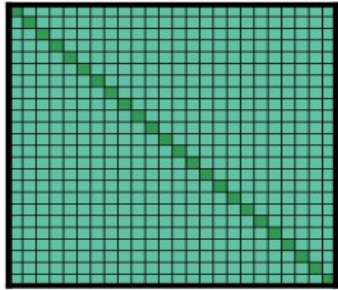
- Key idea: **Chunk input into fixed blocks.**
 - Different heads use different masking matrices.
 - Masking matrices can be any permutation.
- The complexity is reduced from $N \times N$ to $N \times B$.
 - B is the block size (e.g., $N/2$, $N/3$).
- Key Results:



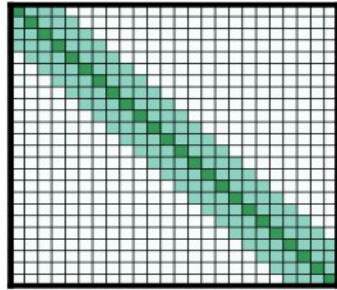
N	Model	Training Time (day)	Memory (per GPU, GB)	Heads Config.	Valid. ppl
512	RoBERTa-1seq	6.62	9.73	-	3.58
	BlockBERT $n=2$	5.83 (-12.0%)	7.91 (-18.7%)	10:2	3.56
	BlockBERT $n=3$	5.80 (-12.5%)	7.32 (-23.8%)	8:2:2	3.71
1024	RoBERTa-1seq	9.66	13.39	-	3.60
	BlockBERT $n=2$	7.51 (-22.3%)	9.73 (-27.3%)	9:3	3.57
	BlockBERT $n=3$	7.23 (-25.1%)	8.55 (-36.1%)	8:2:2	3.63

Sparse Attention — LongFormer

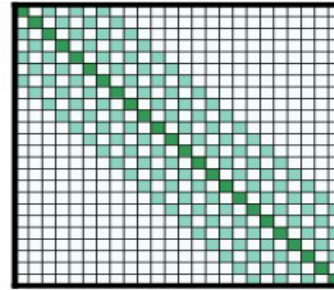
Local Attention + Global Attention



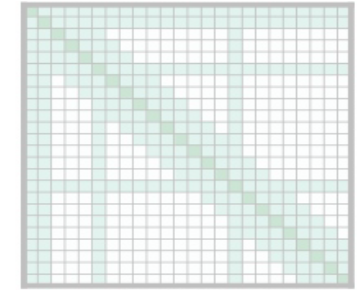
(a) Full n^2 attention



(b) Sliding window attention



(c) Dilated sliding window

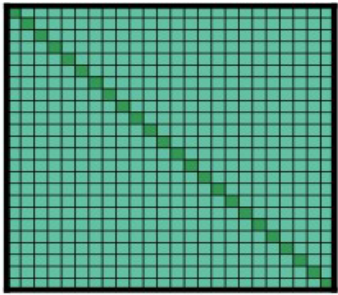


(d) Global+sliding window

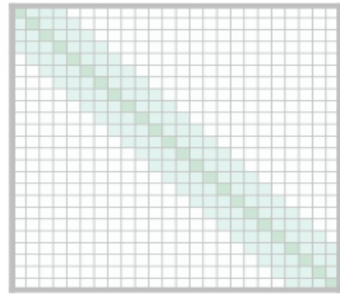
- Attention with **sliding window** (analogous to CNNs):
 - A fixed-size window attention surrounding each token.
 - The complexity is reduced from $O(N^2)$ to $O(N \times W)$, where W is the window size.
- Attention with **dilated sliding window** (analogous to dilated CNNs):
 - Dilate the sliding window with gaps of size dilation D .
 - The receptive field is enlarged from W to $W \times D$, with the same complexity.

Sparse Attention — LongFormer

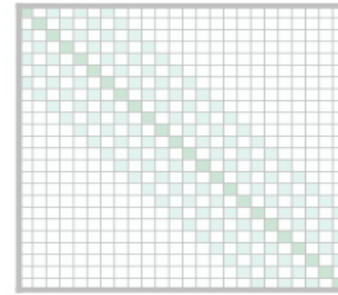
Local Attention + Global Attention



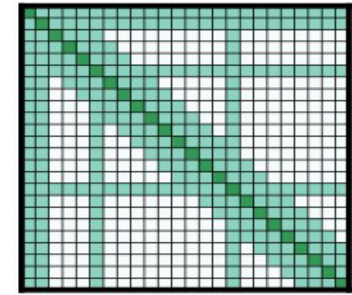
(a) Full n^2 attention



(b) Sliding window attention



(c) Dilated sliding window

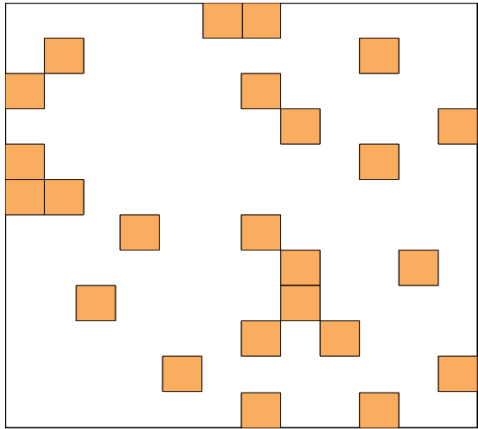


(d) Global+sliding window

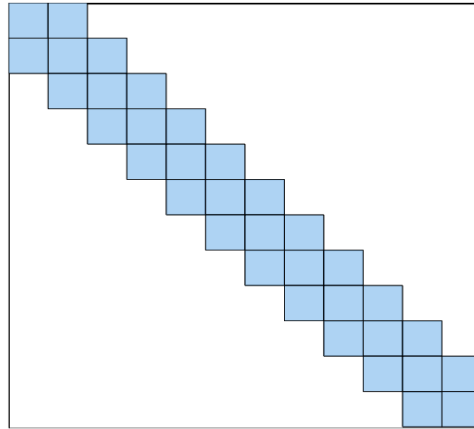
- **Global attention** added on a few **pre-selected** input locations:
 - Classification: The special token ([CLS]), aggregating the whole sequence.
 - QA: All question tokens, allowing the model to compare the question with the document.
- Global attention is applied **symmetrically**:
 - A token with a global attention attends to all tokens across the sequence, and all tokens in the sequence attend to it.

Sparse Attention — Big Bird

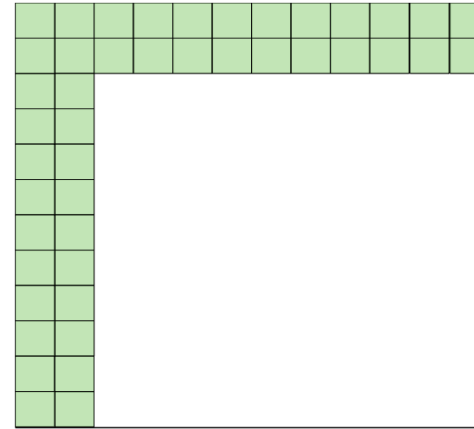
Random Attention + Local Attention + Global Attention



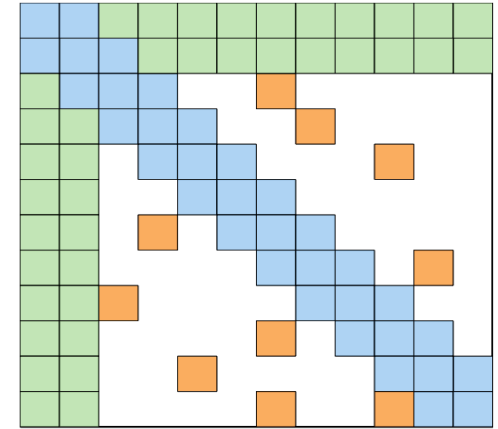
(a) Random attention



(b) Window attention



(c) Global Attention



(d) BIGBIRD

- Random sparse attention:
 - Each query attends over r random number of keys: *i.e.* $A(i, \cdot) = 1$ for r randomly chosen keys.
 - Information can flow fast between any pair of nodes (rapid mixing time for random walks).

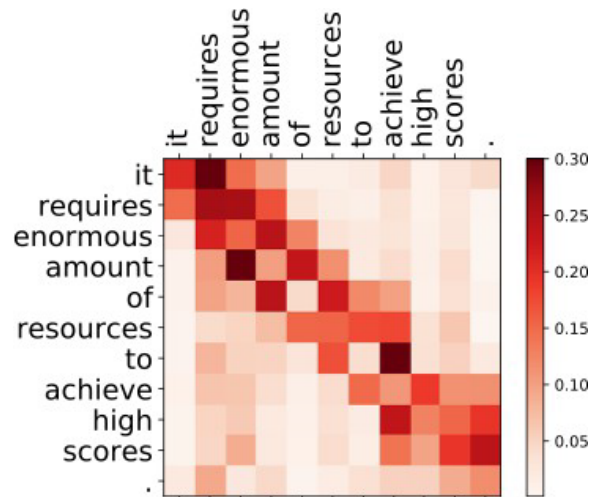
Sparse Attention — Lite Transformer

Local Convolution + Global Attention

- Long-Short Range Attention (LSRA):
 - **Convolution**: Efficiently extract the **local** features.
 - **Attention**: Tailored for **global** feature extraction.

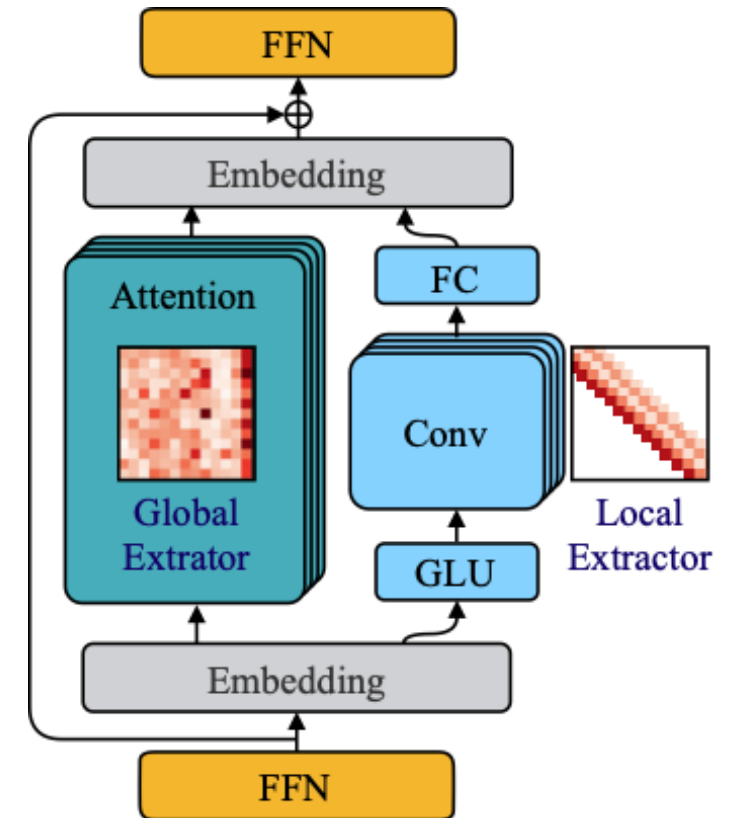
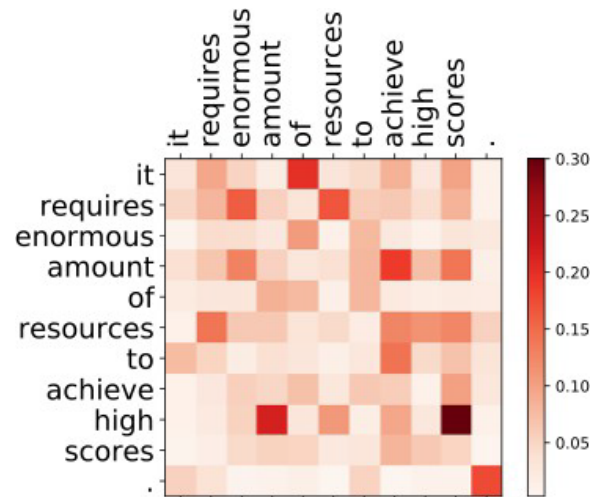
Original Attention

(Too much emphasize on local feature extraction)



Attention in LSRA

(Dedicated for global feature extraction)

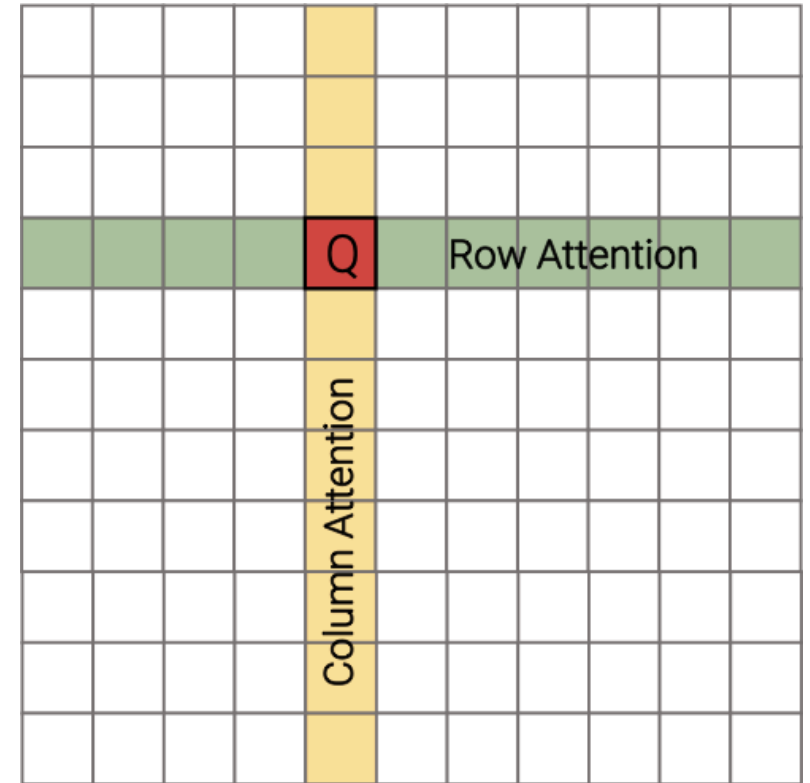


Lite Transformer with Long-Short Range Attention [Wu et al., 2020]

Sparse Attention — Axial Transformer

Row Attention + Column Attention (for 2D Image Recognition)

- The $O(N^2)$ complexity is prohibitive for 2D images:
 - $N = H \times W$, where H and W are height and width.
- Key idea: **Factorize attention along different axes.**
 - Each attention mixes information along a particular axis, while keeping information along other axes **independent**.
 - Analogous to spatially separable convolution:
 - Decompose a 3×3 conv into 1×3 and 3×1 convs.
- The complexity is reduced from $O(H^2W^2)$ to **$O(H^2 + W^2)$** .

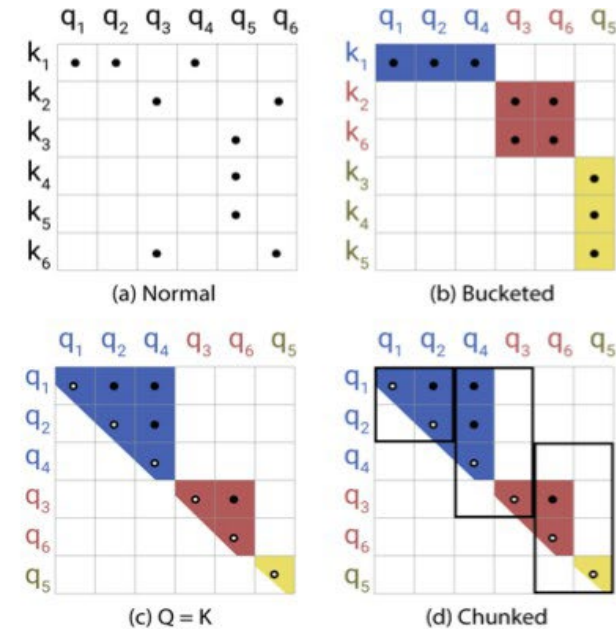
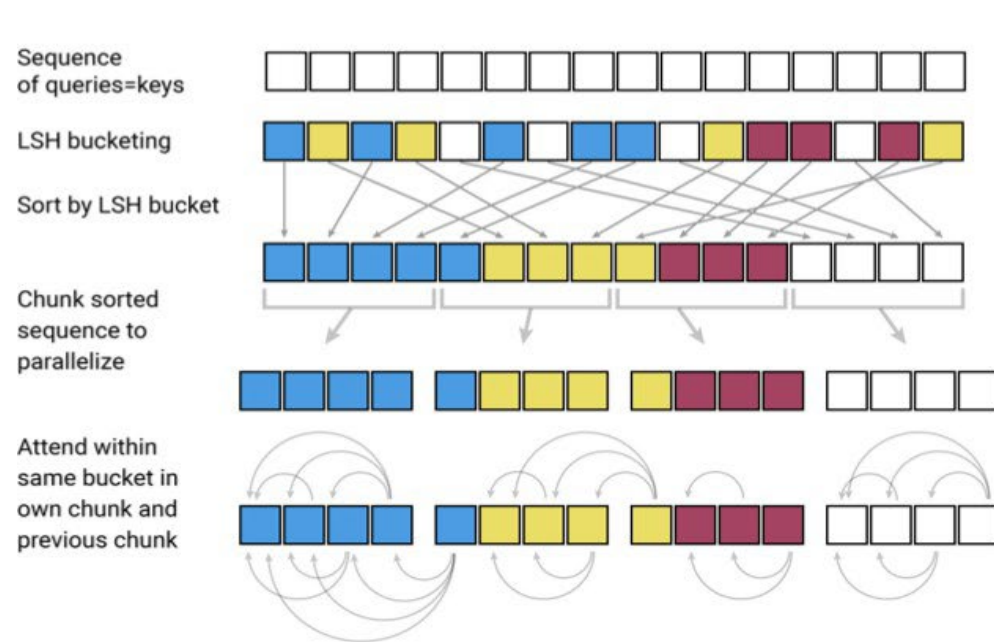


2D Feature Map

(This is not the attention matrix!)

Sparse Attention — Reformer

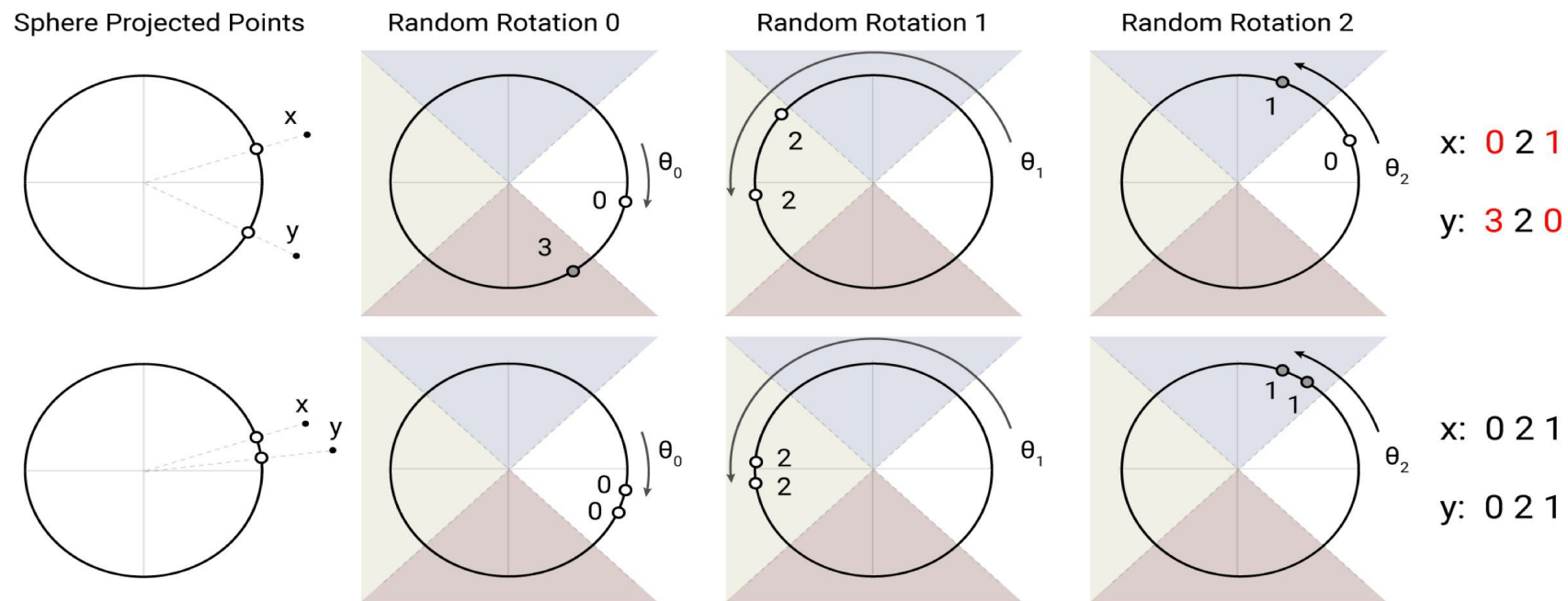
Learned Bucket-wise Attention



- Assumption: **Q** (queries) = **K** (keys)
- Intuition: Attention weight $Q^T K$ is **large** only if Q_i and K_j are **similar**.
- Key idea: **Group similar vectors into the same bucket** (using locality-sensitive hashing).

Sparse Attention — Reformer

Locality-Sensitive Hashing (LSH)



- Use **random rotations** of spherically projected points to establish buckets by an argmax over signed axes projections.

$$h(x) = \arg \max([xR; -xR])$$

Low-Rank Approximations

- Improve efficiency by leveraging **low-rank** approximations of the self-attention matrix.
- The key idea is to assume **low-rank structure** in the $N \times N$ matrix.

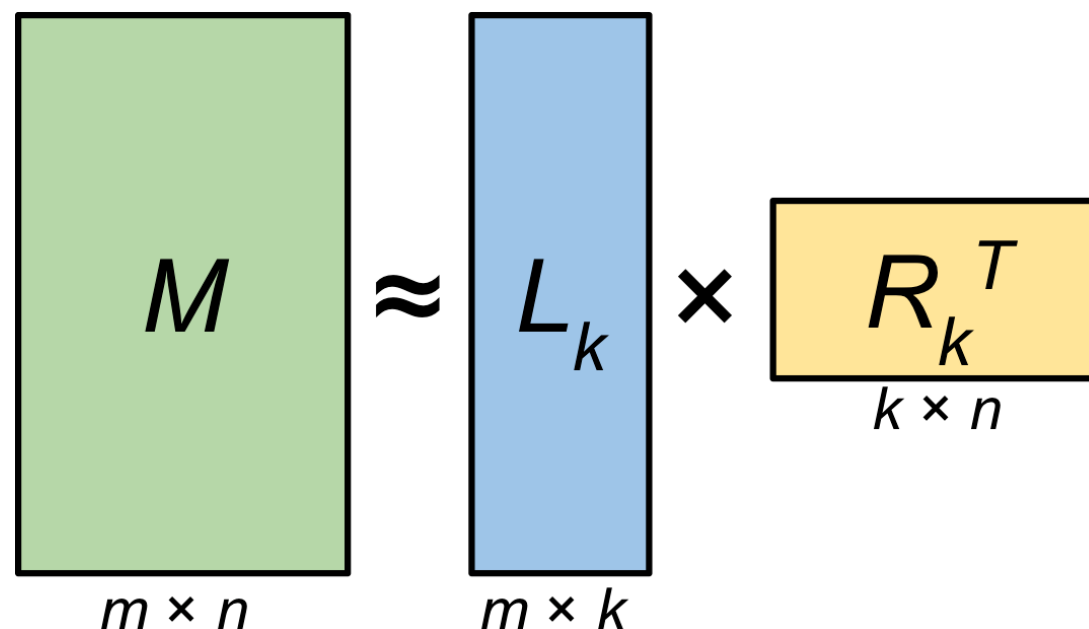
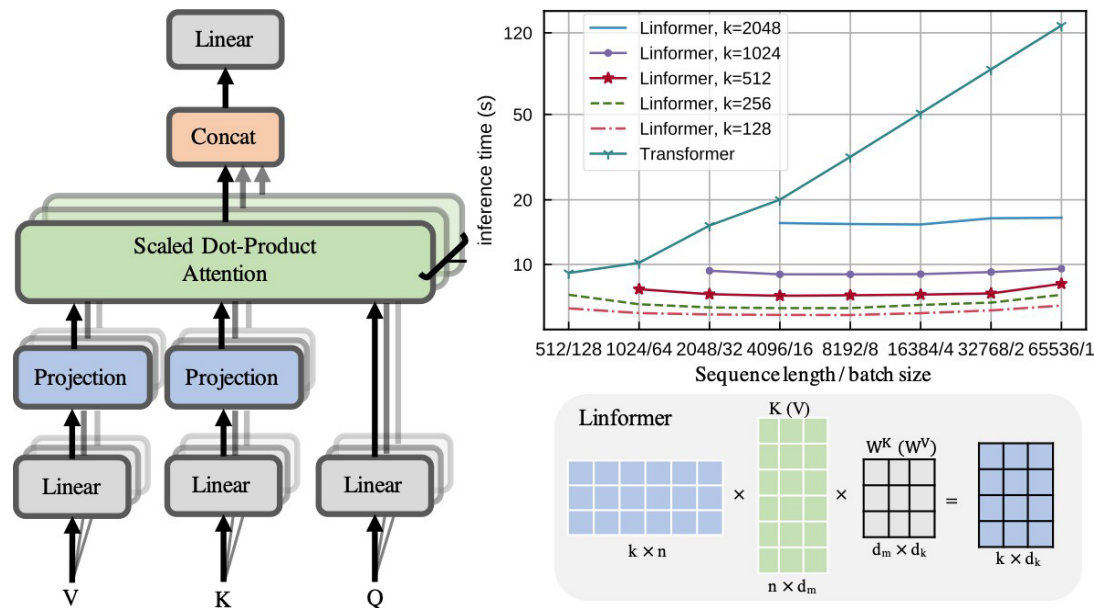

$$\begin{matrix} \boxed{M} \\ m \times n \end{matrix} \approx \begin{matrix} \boxed{L_k} \\ m \times k \end{matrix} \times \begin{matrix} \boxed{R_k^T} \\ k \times n \end{matrix}$$

Image credit: <https://dustinstansbury.github.io/theclevermachine/assets/images/svd-data-compression/low-rank-approximation.png>

Low-Rank Approximations — Linformer

Approximate Self-Attention with Low-Rank Matrix



$$\begin{aligned}\overline{\text{head}}_i &= \text{Attention}(QW_i^Q, \boxed{E_i}KW_i^K, \boxed{F_i}VW_i^V) \\ &= \underbrace{\text{softmax}\left(\frac{QW_i^Q(E_iKW_i^K)^T}{\sqrt{d_k}}\right)}_{\bar{P}: n \times k} \cdot \underbrace{F_iVW_i^V}_{k \times d}\end{aligned}$$

- It projects the **length dimension (not the feature dimension)** of keys and values to a lower-dimensional representation (from N to k).
- Low-rank method reduces the memory complexity problem of self-attention (from N N to N×k).

Linformer: Self-Attention with Linear Complexity [Wang *et al.*, 2020]

Kernelization — Linear Transformer

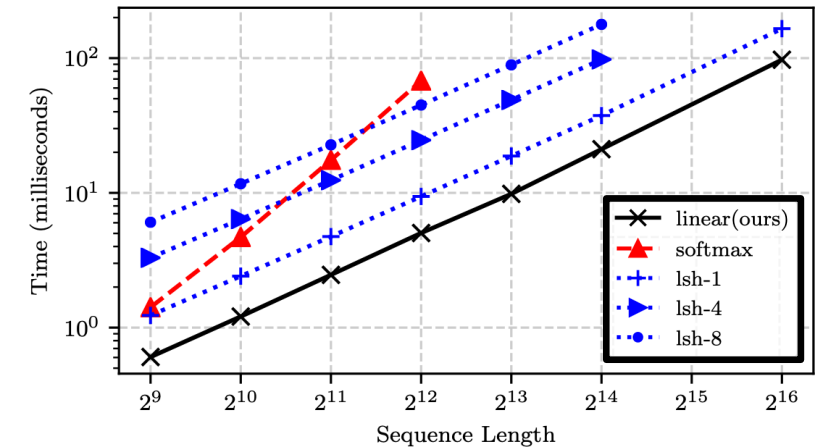
- Generalized attention can be formulated as

$$O_i = \sum_{j=1}^N \frac{\text{Sim}(Q_i, K_j)}{\sum_{j=1}^N \text{Sim}(Q_i, K_j)} V_j, \quad \text{where } Q = xW_Q, K = xW_K, V = xW_V$$

- Softmax attention is a special case with $\text{Sim}(Q, K) = \exp(\frac{QK^T}{\sqrt{d}})$
- Linearized attention is defined with $\text{Sim}(Q, K) = \phi(Q)\phi(K)^T$

$$O_i = \sum_{j=1}^N \frac{\phi(Q_i)\phi(K_j)^T}{\sum_{j=1}^N \phi(Q_i)\phi(K_j)^T} V_j = \frac{\sum_{j=1}^N (\phi(Q_i)\phi(K_j)^T) V_j}{\phi(Q_i) \sum_{j=1}^N \phi(K_j)^T} = \frac{\overset{1 \times D}{\phi(Q_i)} \overset{D \times D}{(\sum_{j=1}^N \phi(K_j)^T V_j)}}{\overset{1 \times D}{\phi(Q_i)} \overset{D \times 1}{(\sum_{j=1}^N \phi(K_j)^T)}} \quad \boxed{\text{Shared across different } O_i}$$

- Complexity is reduced from **O(N²D)** to **O(ND²)**.





ALMA MATER STUDIORUM
UNIVERSITÀ DI BOLOGNA

Credits

Most of this teaching material is assembled from the following teaching material:

MIT 6.S965: TinyML and Efficient Deep Learning Computing by prof. Song Han

Transformer from Scratch, Umar Jamil, <https://github.com/hkproj/transformer-from-scratch-notes>

Transformers, Lucas Beyer, Google Research Brain Team, Zurich